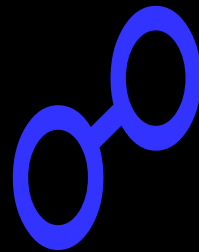


Dezvoltarea aplicațiilor Web



Ontologii în contextul WWW

OWL 2

de la formalizare la realizarea raționamentelor

Prof. Sabin Corneliu Buraga

profs.info.uaic.ro/sabin.buraga/

„O hartă nu înseamnă teritoriul.”

Alfred Korzybski

realități

Ontologiile au drept scop
modelarea unei (părți a unei) lumi

termenii limbajului de modelare folosit
corespund entităților din cadrul lumii

În unele cazuri, modelarea unor domenii
necesită expresivități care nu sunt oferite
de limbajul OWL 1.0

Extinderea OWL cu noi construcții care însă
să păstreze proprietățile din cadrul OWL DL

OWL 2

www.w3.org/TR/owl2-overview/

P. Hitzler *et al.* (Eds.), *OWL 2 Web Ontology Language
Primer* (2nd Edition), W3C, 2012

www.w3.org/TR/owl2-primer/

Exprimarea facilă a reuniunilor disjuncte
*“disjoint union of the class expressions,
all of which are pairwise disjoint”*

:AutomobileDoors owl:disjointUnionOf (
:CarDoor :FrontDoor :RearDoor :TrunkDoor).

// sintaxa funcțională

DisjointUnion (:CarDoor :FrontDoor :RearDoor :TrunkDoor)

Specificarea disjuncției într-o mulțime de clase

```
[ ] rdf:type    owl:AllDisjointClasses ;  
    owl:members ( :ProgrammingLanguage  
                    :MarkupLanguage  
                    :QueryLanguage  
                    :NaturalLanguage  
                    ) .
```

Exprimarea negațiilor privitoare la un individ

// Ion nu s-a născut la oraș

// (negații privind proprietățile de tip obiectual)

```
_:x rdf:type          owl:NegativePropertyAssertion ;  
    owl:sourceIndividual :lon ;  
    owl:assertionProperty schema:birthPlace ;  
    owl:targetIndividual  <http://dbpedia.org/ontology/Town> .
```

Exprimarea negațiilor privitoare la un individ

exemplu: Cristina nu lucrează cu Bogdan – modelare cu
vocabularul *Relationship*: <http://vocab.org/relationship/>

```
_:x a owl:NegativePropertyAssertion ;  
    owl:sourceIndividual :Cristina ;  
    owl:assertionProperty rel:worksWith ;  
    owl:targetIndividual :Bogdan .
```

Exprimarea negațiilor privitoare la un individ

// Anca nu are 33 de ani (negații la nivel de tipuri de date)

```
_:x rdf:type owl:NegativePropertyAssertion ;  
owl:sourceIndividual :Anca ;  
owl:assertionProperty :hasAge ;  
owl:targetValue "33"^^xsd:byte .
```

Restricții de cardinalitate calificate

exemplu: un tată fericit are măcar o fiică
(*i.e.* un copil de gen feminin)

```
:HappyFather rdfs:subClassOf [  
  rdf:type owl:Restriction ;  
  owl:minQualifiedCardinality      "1"^^xsd:byte ;  
  owl:onProperty                    :hasChild ;  
  owl:onClass                       :Female  
].
```

Restricții privind (intervale de) valori

alături de tipurile de date specificate de XML Schema (xsd:byte, xsd:integer, xsd:float, xsd:boolean, xsd:string etc.), sunt definite **owl:real** și **owl:rational**

pot fi utilizate restricțiile (*facets*) XML Schema
e.g., **DatatypeRestriction** (xsd:integer xsd:minInclusive 18)

exemplu: definirea clasei persoanelor tinere

```
:Youngster a schema:Person ;  
  rdfs:subClassOf [  
    rdf:type owl:Restriction ;  
    owl:onProperty :hasAge ;  
    owl:someValuesFrom [  
      rdf:type rdfs:Datatype ;  
      owl:onDatatype xsd:byte ;  
      owl:withRestrictions ( _:min _:max )  
    ] ;  
  ].  
_:min xsd:minInclusive "18"^^xsd:byte .  
_:max xsd:maxInclusive "29"^^xsd:byte .
```

Class: **Youngster**

// sintaxa Manchester

SubClassOf: **hasAge** some byte [≥ 18 , ≤ 29]

Modelări privind reflexivitatea locală

fapt: „narcisiștii se plac pe ei înșiși”

```
SubClassOf ( :Person ObjectHasSelf ( :likes ))
```

Modelări privind reflexivitatea locală

fapt: „narcisiștii se plac pe ei înșiși”

SubClassOf (:Person ObjectHasSelf (:likes))

la nivelul datelor, se poate stipula „Tux este narcisist”:

**:tux a [rdf:type owl:Restriction ;
owl:onProperty :likes ;
owl:hasSelf "true"^^xsd:boolean] .**

Modelarea proprietăților disjuncte

timpul de începere a unui eveniment trebuie să fie diferit
de cel al finalizării acestuia:

schema:startDate owl:propertyDisjointWith schema:endDate .

a avea o diplomă $\neq$ a fi considerat înțelept:

:hasDegree owl:propertyDisjointWith :isConsideredWise .

Posibilitatea de exprimare a cheilor (*keys*), în sensul bazelor de date

exemplificare:

fiecare produs specificat via schema.org are drept cheie
– *i.e.* este identificat unic via – un cod de bare

schema:Product owl:hasKey (:hasBarCode)

Posibilitatea de exprimare a cheilor (*keys*), în sensul bazelor de date

având următoarele:

```
:hasCode a owl:ObjectProperty ;  
    owl:equivalentProperty :hasBarCode .  
:hasBarCode a owl:ObjectProperty ;  
    rdfs:domain owl:Thing ;  
    rdfs:range dbo:Barcode .  
dbo:Barcode a owl:Thing , yago:WikicatStandards .  
:DGilmourLiveAtPompeii :hasCode "0889854649715" .  
:cadou :hasBarCode "0889854649715" .
```

se poate deduce:

```
:DGilmourLiveAtPompeii owl:sameAs :cadou .
```

Posibilitatea de exprimare a cheilor (*keys*),
în sensul bazelor de date

exemplu – cazul FOAF:

foaf:OnlineAccount owl:hasKey

(**foaf:accountName foaf:accountServiceHomepage**)

autentificarea *online* presupune identificarea unică via
numele contului + situl ofertantului unui serviciu digital
(*e.g.*, tux @ blabla.café)

Proprietăți predefinite – de tip obiectual

owl:topObjectProperty

conectează toate perechile posibile de indivizi (obiecte)

owl:bottomObjectProperty

nu conectează nicio pereche de indivizi

Proprietăți predefinite – la nivel de tipuri de date

owl:topDataProperty

conectează toți indivizii posibili cu toți literalii
(instanțe ale tipurilor de date)

owl:bottomDataProperty

nu relaționează niciun individ cu vreun literal

„Înlănțuire” de proprietăți (*property chain*)

specificarea de proprietăți prin compunere,
eventual via axiome

$$hasUncle = hasFather \circ hasBrother$$

```
_:x rdfs:subPropertyOf fam:hasUncle ;  
    owl:propertyChain ( fam:hasFather fam:hasBrother ) .
```

$$grandChildOf = childOf \circ childOf$$

```
        fam:grandChildOf  
owl:propertyChainAxiom ( fam:childOf fam:childOf ) .
```

Suport pentru adnotări

alături de proprietăți între obiecte (*object property*) și cele vizând datele (*data property*), poate fi specificată o suită de proprietăți de adnotare (*annotation property*)

creator, date, versionInfo, priorVersion, backwardCompatibleWith, deprecated, incompatibleWith etc.

Exemplificare: adnotarea explicită a unei axiome

```
:WebDeveloper rdfs:subClassOf :IntelligentPeople .  
[ ] rdf:type owl:Axiom ;  
owl:annotatedSource :WebDeveloper ;  
owl:annotatedProperty rdfs:subClassOf ;  
owl:annotatedTarget :IntelligentPeople ;  
rdfs:comment  
"Orice dezvoltator Web e inteligent."^^xsd:string .
```

WebProtégé Comic Book Ontology Dinosaurus Heterogeneous Computing Infrastructure **Taxonomy Computing Education Terminology**

Classes Properties Individuals Changes By Entity Project Dashboard

Add content to this tab Add tab

Classes

Create Delete Search: Type search s

- owl:Thing
 - ComputingCommunity
 - EducationalProgrammes
 - AlternativeNames
 - Algorithmics
 - Computerology
 - ComputerTechnology
 - Computics
 - Cybernetics
 - ComputerEngineering
 - ComputerScience**
 - Informatics
 - InformationSystems
 - InformationTechnology
 - SoftwareEngineering
 - Industry
 - ComputingIndustry
 - InformationTechnology
 - ProfessionalAssociations
 - ABET
 - ACM

Class description for ComputerScience

Display name
ComputerScience

IRI
<http://webprotege.stanford.edu/RCcmyzocM2s6Glo7QTIJ1vR>

Annotations
rdfs:label ComputerScience

Properties
Meanings Informatics; Information and Communication Technology; Economic Informatics and others
RelatedOccupations computer programmers; computer support specialists; computer systems analysts; software developers; software engineer; software architect

Metrics

Metric	Value
Object Property count	2
Data Property count	3
Annotation Property count	5
Named Individual count	19
Axioms	137
Logical Axioms	31
Annotation axioms	106
OWL 2	Yes

diverse detalii vizând o ontologie
(aici, *Taxonomy Computing Education Terminology*)
oferite de instrumentul **WebProtégé**

Asocierea de profile

stabilirea construcțiilor OWL 2 ce corespund
unei anumite complexități computaționale
în vederea efectuării raționamentelor automate

Profile OWL 2:

raționamente pentru ontologii largi: **EL**
interogări asupra unui volum mare de instanțe: **QL**
raționamente în contextul sistemelor de reguli: **RL**

www.w3.org/TR/owl2-profiles/

de studiat și M. Krötzsch, *OWL 2 Profiles*,
în T. Eiter, T. Krennwallner (eds.),
Reasoning Web Summer School, Springer (2012)
www.korrekt.org/page/OWL_2_Profiles

OWL EL

oferă suport pentru efectuarea de raționamente
în timp polinomial pentru ontologii cu număr mare
de clase și/sau proprietăți
(*e.g.*, cele din domeniul științelor vieții)

facilitează recunoașterea structurilor complexe

OWL EL

nu pot fi folosite:

calificări universale – *e.g.*, **AllValuesFrom**

restricții vizând cardinalitatea

disjuncții la nivel de clase, proprietăți ori obiecte

enumerări ce implică mai mult de un individ

proprietăți obiectuale inverse, (a)simetrice, funcționale

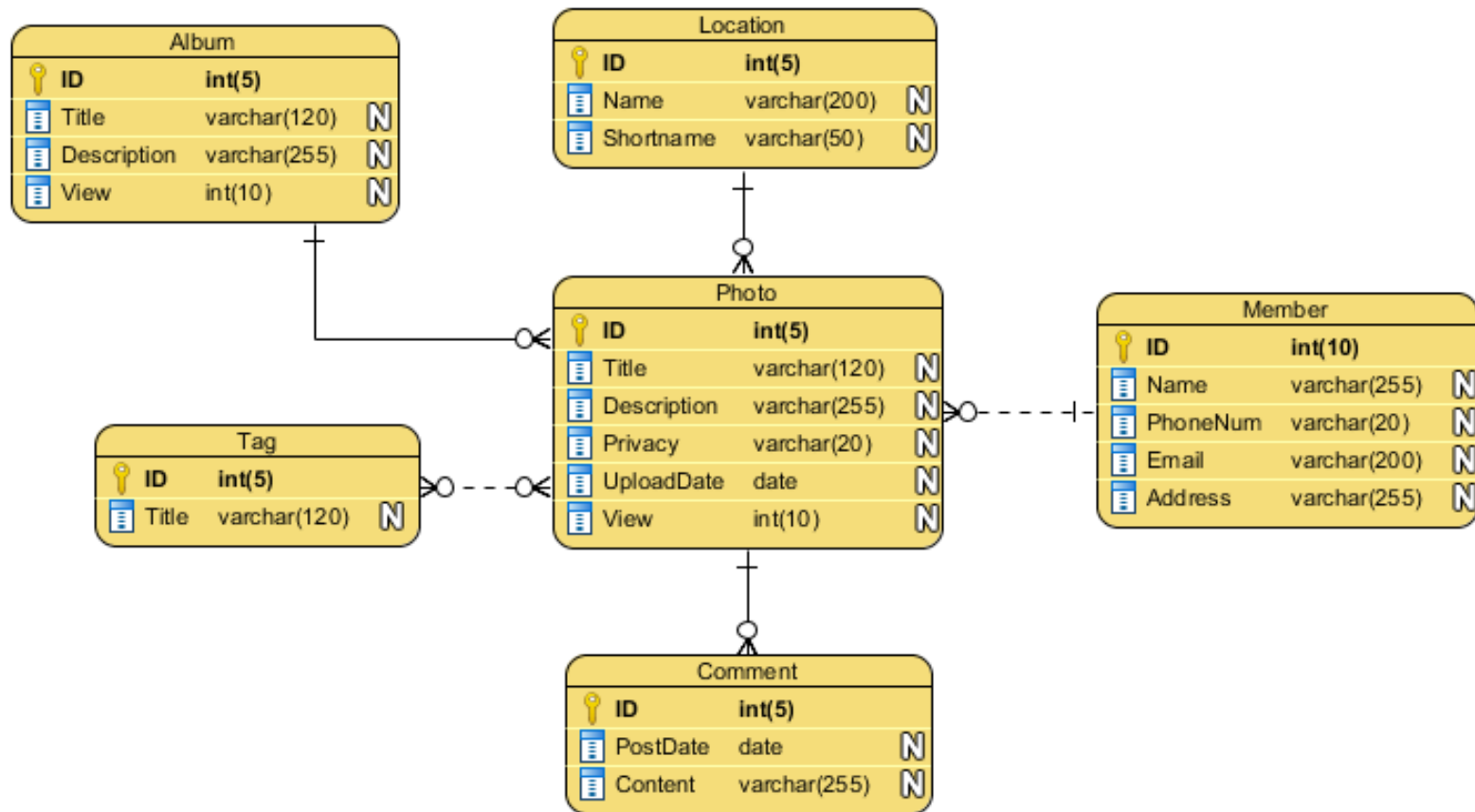
OWL QL

suport pentru efectuarea de interogări (optimizate)
în stilul bazelor de date convenționale



logspace

se pretează pentru ontologii
având un volum mare de fapte (indivizi)



OWL QL oferă expresivitatea necesară specificării de lexicoane sau de scheme de tip UML ori ER (*entity-relation*) similar cu OWL Lite – versiunea 1

OWL QL

se poate baza pe rescrierea interogărilor
conform tehnicilor de raționament automat

OWL QL

nu pot fi utilizate:

calificări existențiale – *e.g.*, **SomeValuesFrom**

calificări universale – *e.g.*, **AllValuesFrom**

enumerări de indivizi/literali

restricții privind cardinalitatea

disjuncția

proprietăți (invers) funcționale și tranzitive

asertiuni privind egalitatea indivizilor (**SameIndividual**)

OWL RL

permite exprimarea regulilor în cadrul ontologiilor

poate fi implementat în cadrul sistemelor convenționale
de baze de date relaționale



complexitate
polinomială

OWL RL

soluție complementară la
limbajele de specificare a regulilor la nivel de Web

RIF – *Rule Interschange Format* (W3C, 2013)

www.w3.org/TR/rif-core/

Specificarea regulilor – de tip *if-then* –
în cadrul **Apache Jena**
(exemplu realizat de Sergiu Bursuc, absolvent FII 2005)

```
# reguli Jena privind descoperirea cunoștințelor despre un regizor
[(?movie cinema:directedBy ?director) ->
  [(?director cinema:directsGenre ?genre) <-
    (?movie cinema:hasGenre ?genre) ]
  [(?director cinema:investigatesTheme ?theme) <-
    (?movie cinema:hasTheme ?theme) ]
  [(?director cinema:usesImagery ?imagery) <-
    (?movie cinema:hasImagery ?imagery) ] ]
```

detalii la jena.apache.org/documentation/inference/

Instrumente oferind suport pentru OWL 2

editoare și/sau medii de dezvoltare:

Apache Jena – jena.apache.org/tutorials/

Protégé – protege.stanford.edu

SimPhoNy – simphony.readthedocs.io

Veda – github.com/semantic-machines/veda

Instrumente oferind suport pentru OWL 2

sisteme de efectuare de raționamente (*reasoners*):

FaCT++

HermiT

Openllet

Pellet

Racer

etc.

Instrumente oferind suport pentru OWL 2

interogare (via OWL QL):

OntoText GraphDB

OpenLink Virtuoso

Oracle Database RDF Graph

Stardog

...și altele

Instrumente oferind suport pentru OWL 2

procesare:

Cowl (C/C++) – swot.sisinflab.poliba.it/cowl/

Horned OWL (Rust) – github.com/phillord/horned-owl

lsw2 (JS + Common Lisp) – github.com/alanruttenberg/lsw2

Ontospy (Python) – lambdamusic.github.io/Ontospy/

OWL API (Java) – owlcs.github.io/owlapi/

Owlready2 (Python) – owlready2.readthedocs.io

pronto (Python) – pronto.readthedocs.io/en/stable/

RDFSharp.Semantics (C#) – github.com/mdesalvo/RDFSharp

Scowl (Scala) – github.com/phenoscape/scowl

...plus, orice bibliotecă oferind suport pentru RDF



În ce manieră exprimăm – formal – înțelesul
(*meaning*) construcțiilor modelate?

entailment

Date o mulțime de enunțuri RDF privitoare la un graf A și o mulțime de enunțuri referitoare la un graf B , atunci A **determină** (*entails*) B dacă fiecare enunț din B este adevărat și în A

entailment

Date o mulțime de enunțuri RDF privitoare la un graf A și o mulțime de enunțuri referitoare la un graf B , atunci A **determină** (*entails*) B dacă fiecare enunț din B este adevărat și în A

aceasta nu oferă obligatoriu seria pașilor deductivi
care conduc de la A la B
(contrast cu procesul de inferență)

entailment

OWL Full *entailment*

OWL DL *entailment*

OWL Lite *entailment*

RDFS *entailment*

RDF *entailment*

util în realizarea interogărilor SPARQL
la nivel de indivizi

www.w3.org/TR/sparql11-entailment/

adaptare după (Hebeler *et al.*, 2009)

Înțelesul (*meaning*) e dat de asocierea unui **formalism**

un model teoretic oferă un mecanism de asociere
de relații între **sintaxă** și **interpretări**

e.g., logica de ordin I (FOL – *First Order Logic*)

necesitatea folosirii unei/unor logici

Pentru o construcție sintactică,
pot exista mai multe sensuri (interpretări, modele)

dorim obiectivitate – *e.g.*, eliminarea ambiguităților

termenul „Java” ≠ termenul „Java”

necesitatea folosirii unei/unor logici

Modelele se presupune că sunt analoage
unei (părți a unei) lumi

elementele modelului corespund obiectelor lumii

necesitatea folosirii unei/unor logici

Trebuie specificată o relație formală
între sintaxă și modele

structura modelelor reflectă relațiile specificate
în cadrul sintaxei

► utilizarea unei/unor logici

necesitatea folosirii unei/unor logici

Logici

limbaje formalizate menite a reprezenta cunoștințe
cu scopul de a putea fi deduse concluzii

necesitatea folosirii unei/unor logici

Logici

sintaxa exprimă propozițiile (*sentences*)
în cadrul limbajului folosit

semantica definește înțelesul – formal – al propozițiilor
i.e., specifică adevărul (*truth*) unei propoziții
în cadrul lumii modelate

necesitatea folosirii unei/unor logici

Exemplu: limbajul aritmetic

$x + 33 > y$ este o propoziție; $x33 + y >$ nu e propoziție

$x + 33 > y$ este adevărată (*true*) *iff*
numărul $x + 33$ nu e mai mic decât numărul y

$x + 33 > y$ este *true* într-o lume în care $x = 1$ și $y = 7$

$x + 33 > y$ este *false* într-o lume în care $x = 1$ și $y = 69$

$x + 33 > x$ este *true* în orice lume – **tautologie**

necesitatea folosirii unei/unor logici

Logicile sunt caracterizate
de ceea ce exprimă (*commit*) ca „primitive”

declarații ontologice – exprimă ce anume există:
fapte (*facts*), lucruri (*things*), timp (*time*), credințe (*beliefs*)

declarații epistemologice – exprimă care este starea
cunoașterii acumulate

Limbaaj (logic)	Declarații ontologice (ce anume există)	Declarații epistemologice (ce cunoaște o entitate/agent)
Logica prop.	fapte (<i>facts</i>)	<i>true / false / unknown</i>
Logica de ordin I (FOL)	fapte, obiecte, relații	<i>true / false / unknown</i>
Logica temporală	fapte, obiecte, relații, timp	<i>true / false / unknown</i>
Teoria probab.	fapte	grade de cunoaștere (<i>belief</i>) 0..1
Logica <i>fuzzy</i>	grade de adevăr	grade de cunoaștere (<i>belief</i>) 0..1

conform Enrico Franconi

necesitatea folosirii unei/unor logici

Modele

lumi având o anumită structură
în care adevărul poate fi evaluat (dedus)

necesitatea folosirii unei/unor logici

Modele

lumi având o anumită structură
în care adevărul poate fi evaluat (dedus)

m este **model** pentru o propoziție p
dacă p este *true* în cadrul modelului m

$M(p)$ reprezintă mulțimea tuturor modelelor lui p

necesitatea folosirii unei/unor logici

Baza de cunoștințe (KB – *knowledge base*)

mulțime de propoziții descrise într-un limbaj formalizat



teorie logică

necesitatea folosirii unei/unor logici

Baza de cunoștințe (KB – *knowledge base*)

conține cunoștințele privitoare la lumea modelată
care pot fi manipulate via **algoritmi deductivi**
(incluși într-un **motor de inferență – *inference engine***)

necesitatea folosirii unei/unor logici

Baza de cunoștințe KB determină, implică, satisface
(*entails*) propoziția p

– adică $KB \models p$ –

dacă și numai dacă

p este *true* în toate lumile în care KB este *true*

dat fiind modelul $M(p)$,

$KB \models p$ dacă și numai dacă $M(KB) \subseteq M(p)$

necesitatea folosirii unei/unor logici

Baza de cunoștințe KB **deduce** (*infer*)
propoziția p folosind algoritmul de inferență i
– adică $KB \vdash_i p$ –
dacă și numai dacă
 p poate fi dedusă (derivată) din KB
de către procedura (algoritmul) i

necesitatea folosirii unei/unor logici

Soundness (corectitudine)

algoritmul i este *sound*

dacă având $KB \vdash_i p$ atunci e adevărat că $KB \models p$

necesitatea folosirii unei/unor logici

Completeness (completitudine)

algoritmul i este *complete*
dacă $KB \models p$ implică faptul că $KB \vdash_i p$

formalizare

Domeniul modelat – partea lumii modelate de ontologie – este interpretat ca o mulțime (*set*) Δ

formalizare

Obiectele (entitățile, *things*) lumii
sunt interpretate ca fiind elemente ale lui Δ

clasele/conceptele (predicate unare)
sunt submulțimi ale lui Δ

proprietățile (predicate binare)
sunt submulțimi ale lui $\Delta \times \Delta = \Delta^2$

predicatele ternare sunt submulțimi ale lui Δ^3

...

formalizare

De exemplu, relația *subClassOf* dintre clase poate fi interpretată ca o incluziune de mulțimi



de revizitat
teoria mulțimilor

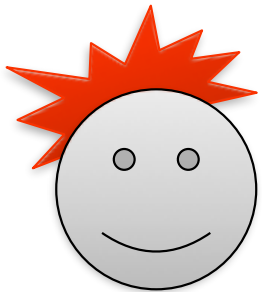
Lume (un web)

Model

Interpretare



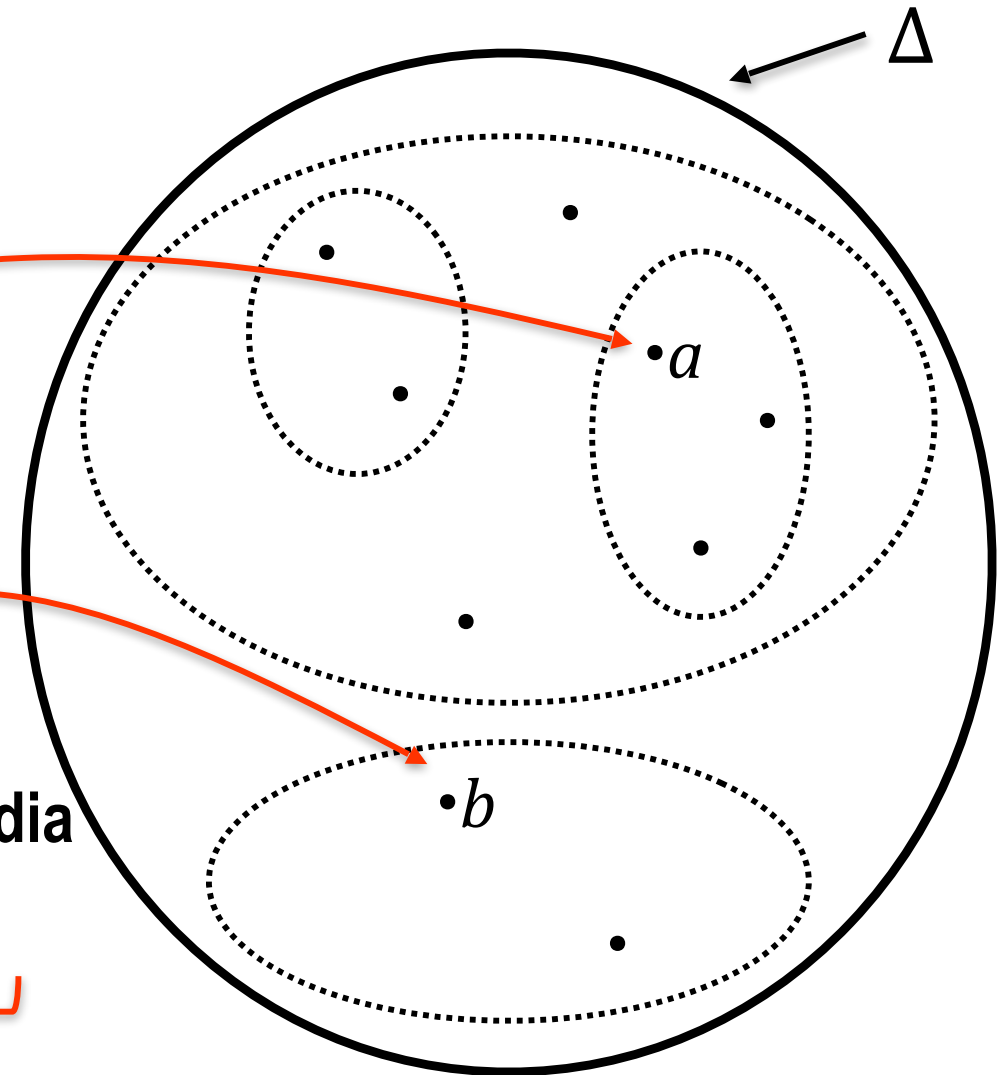
Tux isA Penguin
Penguin kindOf Animal



Anca isA Person
Person kindOf Animal



DGilmourLive isA Multimedia
Anca owns DGilmourLive



$$\{ \langle a, b \rangle , ... \} \subseteq \Delta \times \Delta$$

formalizare

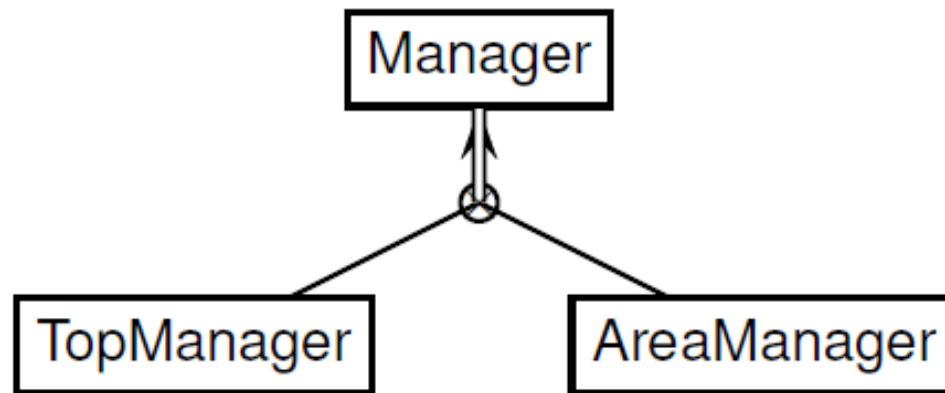
Un **vocabulary** este o mulțime de nume utilizate
în cadrul lumii modelate

{ Tux, Penguin, Animal, Anca, Person, Computer, isA, owns,... }

uzual, exprimat sintactic într-un format deschis
(recurgând la modelul RDF – de dorit)

formalizare

Înțelesul constrângerilor (Enrico Franconi, 2003):



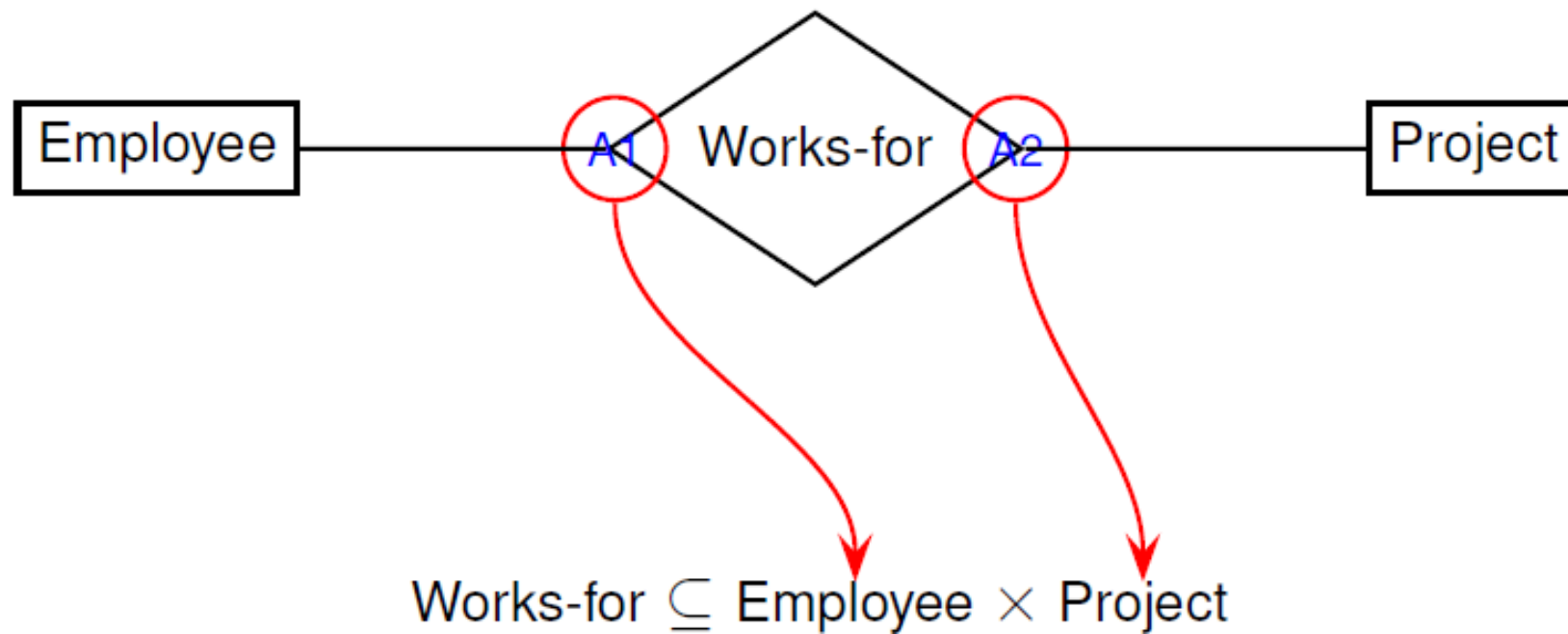
relația de tip *isA*: $\text{AreaManager} \subseteq \text{Manager}$

disjuncția claselor: $\text{AreaManager} \cap \text{TopManager} = \emptyset$

$\text{Manager} \subseteq \text{AreaManager} \cup \text{TopManager}$

formalizare

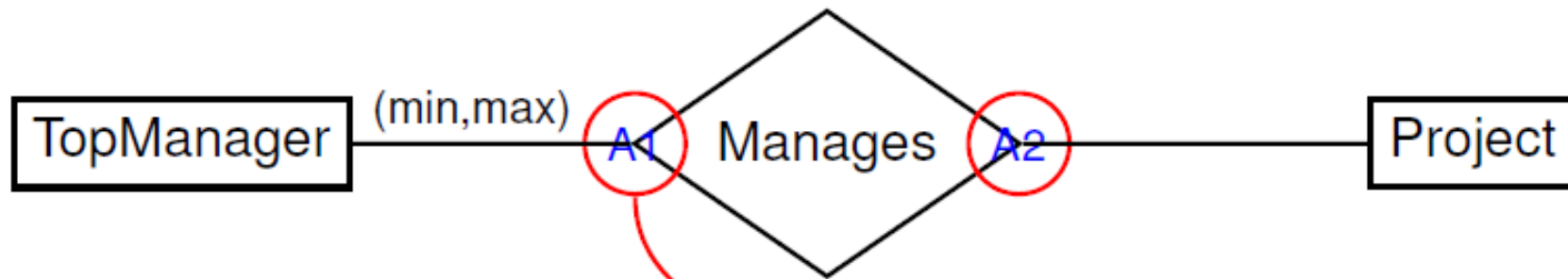
Înțelesul relațiilor (Enrico Franconi, 2003):



:Works-for a owl:**ObjectProperty** ;
rdfs:**domain** :**Employee** ;
rdfs:**range** :**Project** .

formalizare

Înțelesul cardinalităților (Enrico Franconi, 2003):



$$\text{TopManager} \subseteq \{m \mid \max \geq \#(\text{Manages} \cap (\{m\} \times \Omega)) \geq \min\}$$

mulțimea tuturor
instanțelor

formalizare

O interpretare $\mathcal{I}$ a vocabularului e un tuplu $\langle \Delta, \cdot^{\mathcal{I}} \rangle$

domeniul este reprezentat de mulțimea Δ

asocierea dintre sintaxă și semantică e dată de $\cdot^{\mathcal{I}}$

domeniul interpretărilor poate fi infinit

formalizare

O interpretare $\mathcal{I}$ a vocabularului e un tuplu $\langle \Delta, \cdot^{\mathcal{I}} \rangle$

numele obiectelor – asociate elementelor lui Δ

numele predicatelor unare (clase/concepte)

sunt asociate submulțimilor lui Δ

numele predicatelor binare (proprietăți)

sunt asociate submulțimilor $\Delta \times \Delta$

similar, pentru arități superioare – dacă există

formalizare

Ontologiile modelează în special **clase**

formează terminologia

ce trebuie să fie adevărat în legătură cu fiecare concept
din cadrul ontologiei

formalizare

Ontologiile modelează în special **clase**

formează terminologia

ce trebuie să fie adevărat în legătură cu fiecare concept
din cadrul ontologiei

formal: **TBox** – *terminology box*

(schemă ontologică exprimată via RDFS și OWL)

formalizare

Ontologiile oferă un mecanism limitat de exprimare a **indivizilor** – instanțe ale claselor

descrierea indivizilor se poate face
prin baze de date, fișiere CSV, date JSON, documente XML,
scheme de microdate HTML, triple (RDF/RDFa) etc.

formalizare

Ontologiile oferă un mecanism limitat de exprimare a **indivizilor** – instanțe ale claselor

descrierea indivizilor se poate face prin baze de date, fișiere CSV, date JSON, documente XML, scheme de microdate HTML, triple (RDF/RDFa) etc.

formal: **ABox** – *assertions* (datele)

formalizare



Din punct de vedere computațional,
raționamentele privitoare la indivizi
sunt în general dificil/imposibil de realizat

formalizare



Din punct de vedere computațional,
raționamentele privitoare la indivizi
sunt în general dificil/imposibil de realizat

în ipoteza lumilor închise (*closed worlds*),
negația reprezintă eșec – cazul bazelor de date

formalizare: *description logics*

Semantica formală e dată de **logicile descrierii**
(*description logics*)

pentru detalii, de studiat:

F. Baader *et al.* (Eds.), *The Description Logic Handbook: Theory, Implementation and Applications*, Cambridge University Press, 2002

F. Baader *et al.*, *An Introduction to Description Logic*,
Cambridge University Press, 2017 – dltextbook.org

formalizare: *description logics*

Părți decidabile din logica de ordin I
FOL – *First Order Logic*

suport pentru definirea formală de clase și proprietăți
(eventual, pe baza celor deja existente)

pot fi exprimate axiome specificând fapte despre
concepte (clase), roluri (proprietăți) și indivizi (instance)

formalizare: *description logics*

Proprietăți dezirabile în contextul Web-ului
(Ian Horrocks, 2009)

decidabilitate ... important!

complexitate scăzută

formalizare: *reasoning*

Sisteme de inferență (*reasoners*)

algoritmi *sound* și *complete* pentru luarea deciziilor
privind anumite aspecte

pot fi deduse constrângeri suplimentare
e.g., o entitate este sub-entitate a alteia (*subsumption*),
în cazul în care cea de a doua reprezintă
o submulțime a primei entități

formalizare: *reasoning*

Implementări optimizate (pentru OWL 1 și/sau 2)

FaCT++ (C++): bitbucket.org/dtsarkov/factplusplus

HermiT (Java): hermit-reasoner.com

Konclude (C++): github.com/konclude/Konclude

Openllet (Java) bazat pe **Pellet**: github.com/Galigator/openllet

owlrl (Python): github.com/RDFLib/OWL-RL/

Racer (Lisp) – github.com/ha-mo-we/Racer

RDF::Reasoner (Ruby) – github.com/ruby-rdf/rdf-reasoner

SANSA OWL (Scala) – github.com/SANSA-Stack/SANSA-Stack

...

alte instrumente la owl.cs.manchester.ac.uk/tools/list-of-reasoners/

formalizare: *reasoning*

Implementări optimizate pentru OWL 2

clasificatoare pentru ontologii largi (profilul OWL EL):

ELK (Java) – github.com/liveontologies/elk-reasoner

Whelk (Scala) – github.com/balhoff/whelk

vezi și G. Singh *et al.*, “OWL2Bench: A Benchmark for OWL 2 Reasoners”, *International Semantic Web Conference – ISWC 2020*

sumitbhatia.net/papers/iswc20.pdf

github.com/kracr/owl2bench

description logics

Familie de formalisme logice
folosite pentru reprezentarea cunoștințelor

M. Krötzsch *et al.*, *A Description Logic Primer* (2013)
arxiv.org/abs/1201.4089

aspecte fundamentale ale logicii matematice în
Richard Hammack, *Book of Proof* (3rd Edition), 2018:
richardhammack.github.io/BookOfProof/

description logics

Cea mai simplă DL e ALC (încisă propozițional)

Attributive Language with Complements

AL specificată de (Schmidt-Schauß & Smolka, 1991)

description logics

Cea mai simplă DL e $\mathcal{ALC}$ (închisă propozițional)

concepte – eventual, atomice – construite
folosind booleeni $\sqcap, \sqcup, \neg$
plus cuantificatorii $\forall, \exists$

rolurile (proprietățile) pot fi atomice

$$C, D ::= A \mid \top \mid \perp \mid C \sqcap D \mid C \sqcup D \mid \neg C \mid \exists r.C \mid \forall r.C$$

description logics

Exemplificare: „tații fericiți” (Ian Horrocks)

HappyFather $\equiv$ **Man** $\sqcap$
 $\exists$ **hasChild.Female** $\sqcap$ $\exists$ **hasChild.Male** $\sqcap$
 $\forall$ **hasChild**.(**Rich** $\sqcup$ **Happy**)

un tată fericit este un bărbat
care are drept copii – bogați sau fericiți –
măcar o fată și un băiat

description logics – **exemplu concret**

Specificarea unor concepte privind organismul uman
(Ian Horrocks)
context: *e-medicine*

Heart $\sqsubseteq$ **MuscularOrgan** $\sqcap \exists$ **isPartOf.CirculatorySystem**

HeartDisease $\equiv$ **Disease** $\sqcap \exists$ **affects.Heart**

VascularDisease $\equiv$ **Disease** $\sqcap$
 $\exists$ **affects.(isPartOf.CirculatorySystem)**

description logics – **exemplu concret**

Concepte vizând sistemele bazate pe evenimente
(Riccardo Tommasini *et al.*)

context: *distributed computing, IoT – Internet of Things*

TemperatureEvent $\equiv$ **Observation** $\sqcap$
 $\exists$ **observedProperty**.**Temperature**

AbnormalTempEvent $\sqsubseteq$ **TemperatureEvent**

ServerRoomTempEvent $\sqsubseteq$ **AbnormalTempEvent** $\sqcap$
 $(\exists$ **observationResult**.**[hasValue** > 40]) $\sqcap$
 $\exists$ **hasLocation**.**ServerRoom**)

description logics – extensii

S proprietățile sunt tranzitive

H ierarhia proprietăților – *e.g.*, *hasDaughter* $\sqsubseteq$ *hasChild*

O nominali/*singletons* – *e.g.*, { *Tux* }

I proprietăți inverse – *e.g.*, *isChildOf* $\equiv$ *hasChild*-

N restricții de cardinalitate – *e.g.*, ≥ 2 *hasChild*, ≤ 3 *hasChild*

Q restricții de cardinalitate calificate – *e.g.*, ≥ 2 *hasChild.Male*

F restricții de cardinalitate funcționale – *e.g.*, ≤ 1 *hasMother*

„specii” DL ce pot fi, la nivel sintactic, exprimate prin OWL



Complexity of reasoning in Description Logics

Note: the information here is (always) incomplete and [updated](#) often

Base description logic: *Attributive Language with Complements*

$ALC ::= \perp \mid T \mid A \mid \neg C \mid C \cap D \mid C \cup D \mid \exists R.C \mid \forall R.C$



Concept constructors:

- ☐ F – functionality²: $(\leq 1 R)$
 - ☐ N – (unqualified) number restrictions: $(\geq n R)$, $(\leq n R)$
 - ☒ Q – qualified number restrictions: $(\geq n R.C)$, $(\leq n R.C)$
 - ☒ O – nominals: $\{a\}$ or $\{a_1, \dots, a_n\}$ ("one-of")
-
- ☐ μ – least fixpoint operator: $\mu X.C$

complex roles⁵ in number restrictions⁶

TBox (concept axioms) is *internalizable* in extensions of $ALCIO$, see [82, Lemma 4.12], [61, p.3]

- ☒ empty TBox
- ☐ acyclic TBox ($A \equiv C$, A is a concept name; no cycles)
- ☐ general TBox ($C \subseteq D$, for arbitrary concepts C and D)

Role constructors:

- ☒ I – role inverse: R^-
-
- ☐ $\cap$ – role intersection³: $R \cap S$
 - ☐ $\cup$ – role union: $R \cup S$
 - ☐ $\neg$ – role complement: $\neg R$
 - ☐ $\circ$ – role chain (composition): $R \circ S$
 - ☐ $*$ – reflexive-transitive closure⁴: R^*
 - ☐ id – concept identity: $id(C)$

RBox (role axioms):

- ☒ $\mathcal{S}$ – role transitivity: $\text{Tr}(R)$
- ☒ $\mathcal{H}$ – role hierarchy: $R \subseteq S$
- ☐ $\mathcal{R}$ – complex role inclusions: $R \circ S \subseteq R$, $R \circ S \subseteq S$
- ☐ s – some additional features (click to see them)

You have selected a Description Logic: $\mathcal{SHOIQ}$

pentru anumite „specii” DL, pot fi demonstrate diverse proprietăți (*e.g.*, vizând complexitatea timp/spațiu)

vezi Evgeny Zolin – www.cs.man.ac.uk/~ezolin/dl/

description logics

Baza de cunoștințe (KB) e compusă din
2 mulțimi de axiome:

TBox descrie structura domeniului – schema conceptuală

ABox descrie o situație concretă – datele, instanțele

description logics

Baza de cunoștințe (KB) e compusă din
2 mulțimi de axiome:

TBox descrie structura domeniului – schema conceptuală

$\text{SuperStud} \sqsubseteq \text{WebDeveloper} \sqcap \text{Masterand} \sqcap \text{Eminent}$

$\text{HappyFather} \equiv \text{Man} \sqcap \exists \text{hasChild.Female} \sqcap \dots$

transitiveObjectProperty (rudaCu), $\exists \text{rudaCu}.\top \sqsubseteq \text{Persoana}$

ABox descrie o situație concretă – datele, instanțele

$\text{Radu} : \text{HappyFather}, \text{Andreea} : \text{SuperStud}$

$\langle \text{Radu}, \text{Andreea} \rangle : \text{hasChild}$

Knowledge Base

TBox (*schema*)

Man $\equiv$ Human $\sqcap$ Male

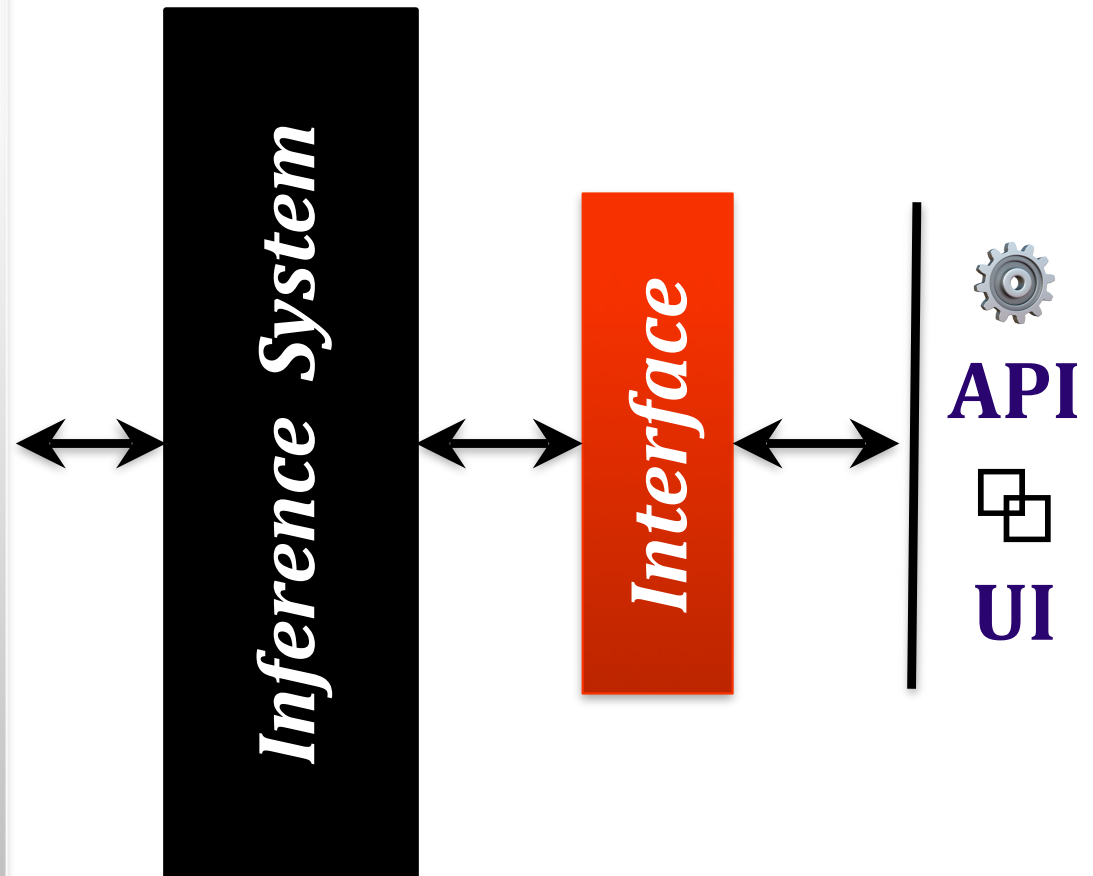
HappyFather $\equiv$ Man $\sqcap$

$\exists$ hasChild.Female $\sqcap$...

ABox (*data*)

Radu : HappyFather

<Radu, Andreea> : hasChild



description logics

Remarcă:

această separație nu are neapărat
o semnificație logică, dar este convenabilă
atât din punct de vedere conceptual,
cât și din cel al implementării

description logics

Constructorii DL privitori la concepte (logica $\mathcal{AL}$):

Construct	Syntax	Example	Semantics
atomic concept	A	Doctor	$A^{\mathcal{I}} \subseteq \Delta^{\mathcal{I}}$
atomic role	P	hasChild	$P^{\mathcal{I}} \subseteq \Delta^{\mathcal{I}} \times \Delta^{\mathcal{I}}$
atomic negation	$\neg A$	$\neg$ Doctor	$\Delta^{\mathcal{I}} \setminus A^{\mathcal{I}}$
conjunction	$C \sqcap D$	Hum $\sqcap$ Male	$C^{\mathcal{I}} \cap D^{\mathcal{I}}$
(unqual.) exist. res.	$\exists R$	$\exists$ hasChild	$\{ o \mid \exists o'. (o, o') \in R^{\mathcal{I}} \}$
value restriction	$\forall R.C$	$\forall$ hasChild.Male	$\{ o \mid \forall o'. (o, o') \in R^{\mathcal{I}} \rightarrow o' \in C^{\mathcal{I}} \}$
bottom	$\perp$		$\emptyset$

A, C, D – concepte (clase)

P, R – roluri (proprietăți)

description logics

O ontologie OWL se poate asocia unei baze de cunoștințe DL notată $\mathcal{K} = \langle \mathcal{T}, \mathcal{A} \rangle$

$\mathcal{T}$ **TBox** – mulțime de axiome de forma:

$C \sqsubseteq D$	incluziunea claselor (conceptelor)
$C \equiv D$	echivalența claselor
$P \sqsubseteq Q$	incluziunea proprietăților (rolurilor)
$P \equiv Q$	echivalența proprietăților
$P^+ \sqsubseteq P$	tranzitivitatea proprietăților

$\mathcal{A}$ **ABox** – mulțime de axiome de forma:

$x \in C$	instanțierea unui concept/clase
$\langle x, y \rangle \in P$	instanțierea unei proprietăți

OWL construct	DL construct	Example
ObjectIntersectionOf	$C_1 \sqcap \dots \sqcap C_n$	Human $\sqcap$ Male
ObjectUnionOf	$C_1 \sqcup \dots \sqcup C_n$	Doctor $\sqcup$ Lawyer
ObjectComplementOf	$\neg C$	$\neg$ Male
ObjectOneOf	$\{a_1\} \sqcup \dots \sqcup \{a_n\}$	{john} $\sqcup$ {mary}
ObjectAllValuesFrom	$\forall P.C$	$\forall$ hasChild.Doctor
ObjectSomeValuesFrom	$\exists P.C$	$\exists$ hasChild.Lawyer
ObjectMaxCardinality	$(\leq n P)$	$(\leq 1$ hasChild)
ObjectMinCardinality	$(\geq n P)$	$(\geq 2$ hasChild)

construcții OWL vs. DL pentru specificarea unui TBox
(Lembo & Rosati, 2009)

pentru OWL, se pot folosi și tipurile de date XML Schema
un exemplu: $\exists$ arePunctaj.byte

OWL axiom	DL syntax	Example
SubClassOf	$C_1 \sqsubseteq C_2$	Human $\sqsubseteq$ Animal $\sqcap$ Biped
EquivalentClasses	$C_1 \equiv C_2$	Man $\equiv$ Human $\sqcap$ Male
DisjointClasses	$C_1 \sqsubseteq \neg C_2$	Man $\sqsubseteq \neg$ Female
SameIndividual	$\{a_1\} \equiv \{a_2\}$	{presBush} $\equiv$ {G.W.Bush}
DifferentIndividuals	$\{a_1\} \sqsubseteq \neg\{a_2\}$	{john} $\sqsubseteq \neg\{peter\}$
SubObjectPropertyOf	$P_1 \sqsubseteq P_2$	hasDaughter $\sqsubseteq$ hasChild
EquivalentObjectProperties	$P_1 \equiv P_2$	hasCost $\equiv$ hasPrice
InverseObjectProperties	$P_1 \equiv P_2^-$	hasChild $\equiv$ hasParent $^-$
TransitiveObjectProperty	$P^+ \sqsubseteq P$	ancestor $^+$ $\sqsubseteq$ ancestor
FunctionalObjectProperty	(functional P)	(functional hasFather)

specificarea axiomelor în sintaxa OWL și, respectiv, DL
(Lembo & Rosati, 2009)

description logics

Axiomele TBox pot include definiții (parțiale)

specificarea claselor atomice (de sine-stătătoare)

specificarea categoriilor de entități
din cadrul unui domeniu – un *web*:
 $C \sqsubseteq D$ sau $C \equiv D$ (adică $C \sqsubseteq D$ și $D \sqsubseteq C$)
unde C și D reprezintă nume de clase

incluziunea conceptelor generale
(*General Concept Inclusion* – GCIs)

description logics

O interpretare $\mathcal{I}$ satisface – modelează – o axiomă A

$\mathcal{I} \models A$ dacă:

$$\text{TBox} \left\{ \begin{array}{ll} \mathcal{I} \models C \sqsubseteq D \text{ iff } C^{\mathcal{I}} \subseteq D^{\mathcal{I}} & \mathcal{I} \models C \equiv D \text{ iff } C^{\mathcal{I}} = D^{\mathcal{I}} \\ \mathcal{I} \models R \sqsubseteq S \text{ iff } R^{\mathcal{I}} \subseteq S^{\mathcal{I}} & \mathcal{I} \models R \equiv S \text{ iff } R^{\mathcal{I}} = S^{\mathcal{I}} \\ \mathcal{I} \models R^+ \sqsubseteq R \text{ iff } (R^{\mathcal{I}})^+ \subseteq R^{\mathcal{I}} & \end{array} \right.$$

$$\text{ABox} \left\{ \begin{array}{l} \mathcal{I} \models x \in D \text{ iff } x^{\mathcal{I}} \in D^{\mathcal{I}} \\ \mathcal{I} \models \langle x, y \rangle \in R \text{ iff } (x^{\mathcal{I}}, y^{\mathcal{I}}) \in R^{\mathcal{I}} \end{array} \right.$$

description logics

Definiții privind satisfiabilitatea:

I satisface mulțimea TBox $\mathcal{T}$ (adică $I \models \mathcal{T}$)

iff *I* satisface orice axiomă A din $\mathcal{T}$

I satisface mulțimea ABox $\mathcal{A}$ ($I \models \mathcal{A}$)

iff *I* satisface orice axiomă A din $\mathcal{A}$

I satisface baza de cunoștințe $\mathcal{K}$ ($I \models \mathcal{K}$)

iff *I* satisface și $\mathcal{T}$ și $\mathcal{A}$

description logics

Cunoștințele sunt semnificative (*meaningful*)

clasele pot avea instanțe:

conceptul C este satisfiabil în ceea ce privește $\mathcal{K}$
iff există *un anumit model* I al lui $\mathcal{K}$ astfel încât $C^I \neq \emptyset$

description logics

Cunoștințele sunt **corecte** – modelează intuițiile:

C subsumează (*subsumption*) D – adică $C \sqsubseteq D$ –

în ceea ce privește $\mathcal{K}$

iff pentru **orice model** I al lui $\mathcal{K}$, $C^I \subseteq D^I$

description logics

Cunoștințele sunt *minimal redundante*
– nu există sinonime nedorite:

C este *echivalent* (*equivalent*) cu D ($C \equiv D$)
în ceea ce privește $\mathcal{K}$

iff pentru *orice model* I al lui $\mathcal{K}$, $C^I = D^I$

description logics

Interogarea cunoștințelor:

x este o instanță a conceptului C în ceea ce privește $\mathcal{K}$
iff pentru *orice model* I al lui $\mathcal{K}$, $x^I \in C^I$

$\langle x, y \rangle$ este o instanță a rolului (proprietății) P
în ceea ce privește $\mathcal{K}$
iff pentru *orice model* I al lui $\mathcal{K}$, $(x^I, y^I) \in P^I$

description logics

Aspectele anterioare sunt reductibile la
consistența bazei de cunoștințe

o bază de cunoștințe K este consistentă
iff există *un anumit model* I al lui K

consistența bazei de cunoștințe este reductibilă la
consistența conceptelor (*concept consistency*)

description logics

Verificarea formală a consistenței e utilă pentru
proiectarea și mentenanța de ontologii

semnificative – toate clasele pot avea indivizi

corecte – exprimă intuițiile experților domeniului

minimal redundante – nu există sinonime nedorite

axiomatizate – există (suficiente) descrieri detaliate

conform lui Ian Horrocks

description logics

Verificarea formală a consistenței e utilă pentru

oferirea de răspunsuri privind clasele/indivizii

găsirea claselor mai generale/particulare

extragerea de indivizi conform unei interogări date

conform lui Ian Horrocks

description logics

Pentru verificarea satisfiabilității (consistenței)
se pot utiliza **algoritmi de tip tablou**
(hyper-)tableau algorithms

B. Glimm, Y. Kazakov, “Classical Algorithms for Reasoning and
Explanation in Description Logics”,
Reasoning Web. Explainable Artificial Intelligence, Springer, 2019
www.uni-ulm.de/fileadmin/website_uni_ulm/iui.inst.090/Publikationen/2019/GliKaz19Classical.pdf

lucrări de interes via Google Scholar:
scholar.google.com/scholar?q=hyper-tableau+algorithms

description logics

Dat fiind un concept C ,
se încearcă materializarea unui model
(exemplu concret) arborescent consistent cu axiomele
din baza de cunoștințe – faptele de bază din ABox

description logics

Conceptul C este descompus la nivel sintactic
se folosesc conceptele complexe și axiomele din TBox

description logics

Se aplică regulile de expandare a tabloului
(*tableau expansion rules*)

se deduc constrângerile asupra elementelor modelului

description logics

Regulile de tip tablou
corespund constructorilor din logică

de exemplu: $\sqcap$, $\sqcup$ etc.

unele reguli sunt *nedeterminate* – *e.g.*, $\sqcup$

în practică, aceasta înseamnă *căutare*

description logics

Ne oprim când nu mai pot fi aplicate reguli ori
apare un **conflict** (*clash*)

conflictul reprezintă o contradicție evidentă

de exemplu: $A(x), \neg A(x)$

pentru terminare, poate fi necesară
verificarea ciclurilor (blocarea – *blocking*)

description logics

C este satisfiabil dacă și numai dacă regulile
pot fi aplicate astfel încât este construit
un arbore complet expandat fără conflicte

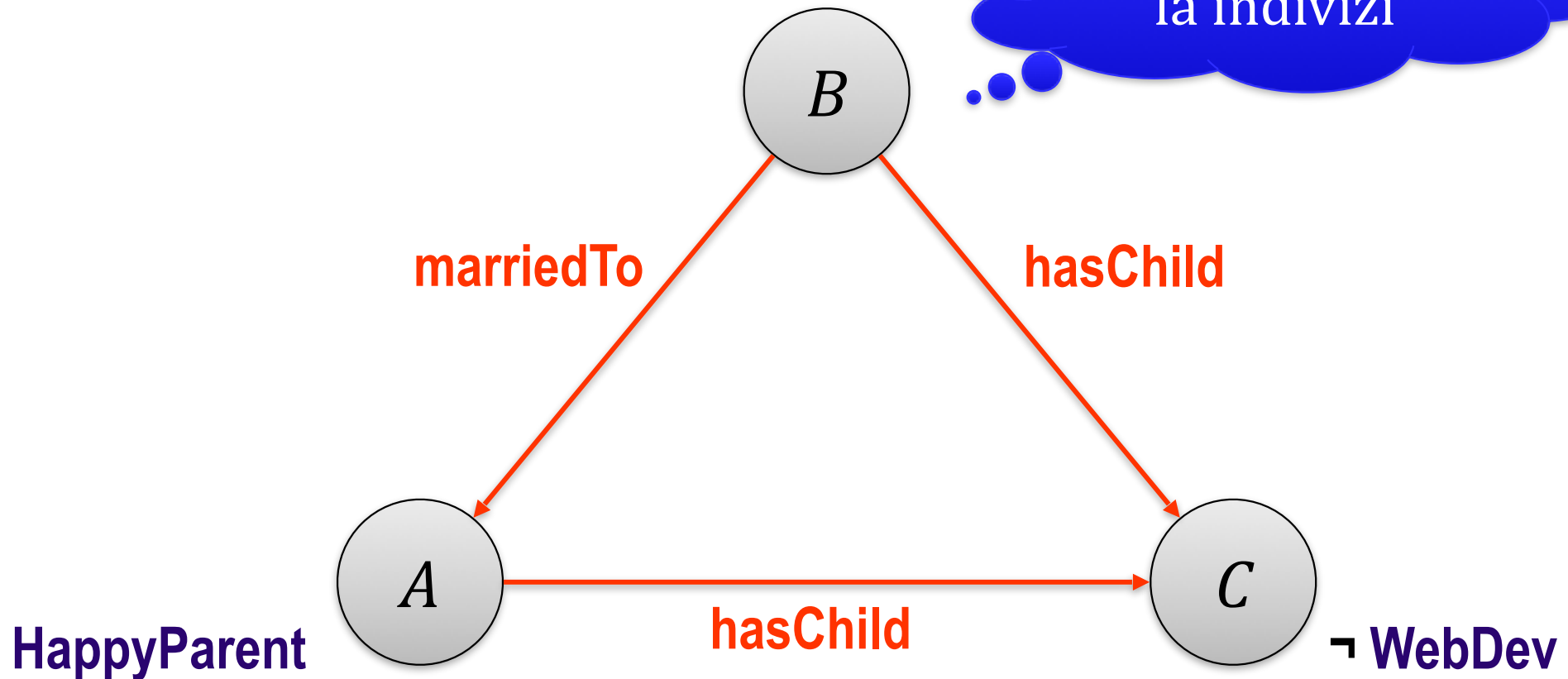
description logics – exemplu

Fie baza/graful de cunoștințe

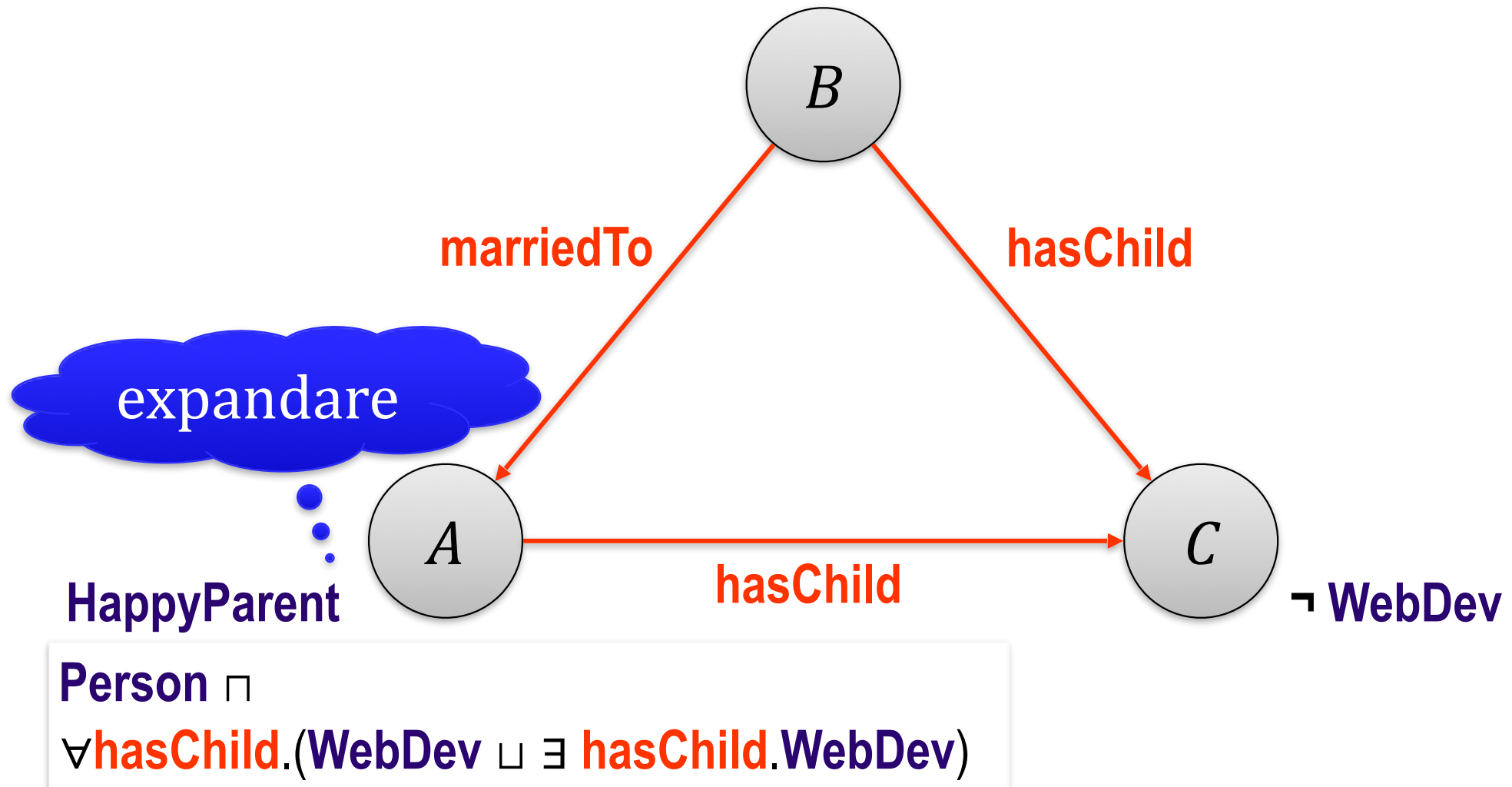
$$\{ \text{HappyParent} \equiv \text{Person} \sqcap$$
$$\forall \text{hasChild} . (\text{WebDev} \sqcap \exists \text{hasChild} . \text{WebDev}),$$
$$\text{Alex: HappyParent}, \text{Alex hasChild Cristina},$$
$$\text{Cristina: } \neg \text{WebDev},$$
$$\text{Bianca hasChild Cristina},$$
$$\text{Bianca marriedTo Alex} \}$$

description logics – exemplu

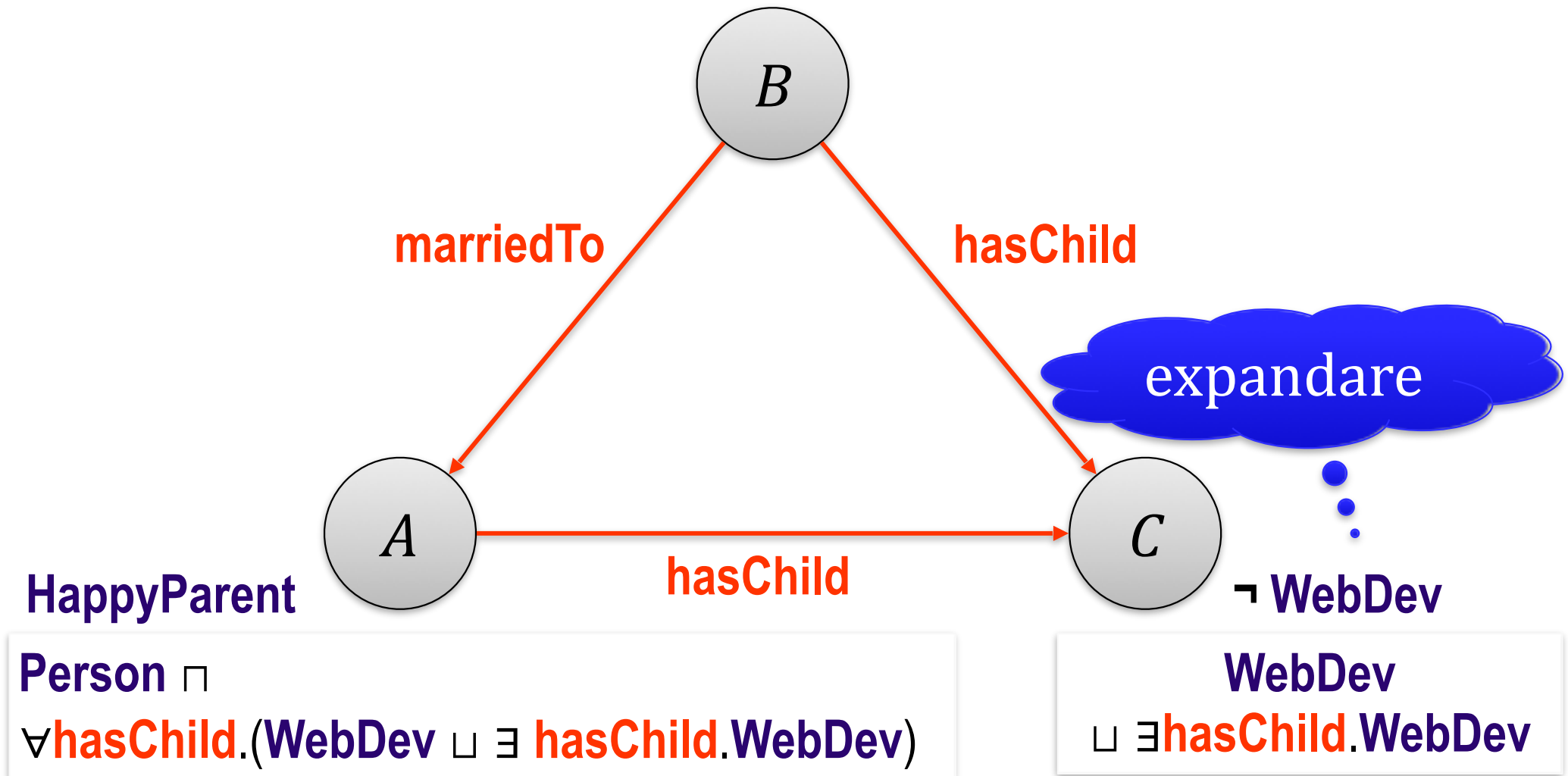
fapte privitoare
la indivizi



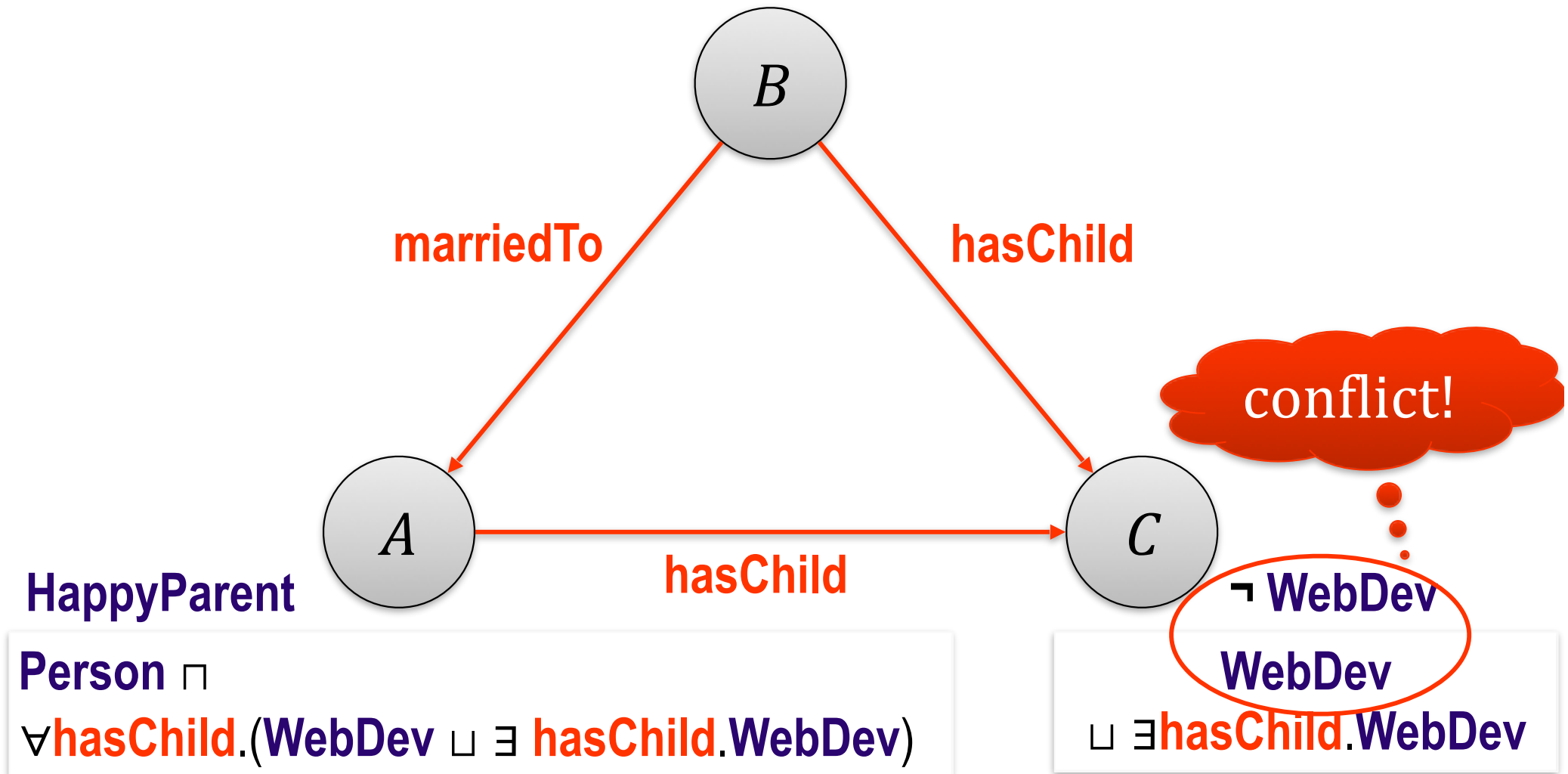
description logics – exemplu



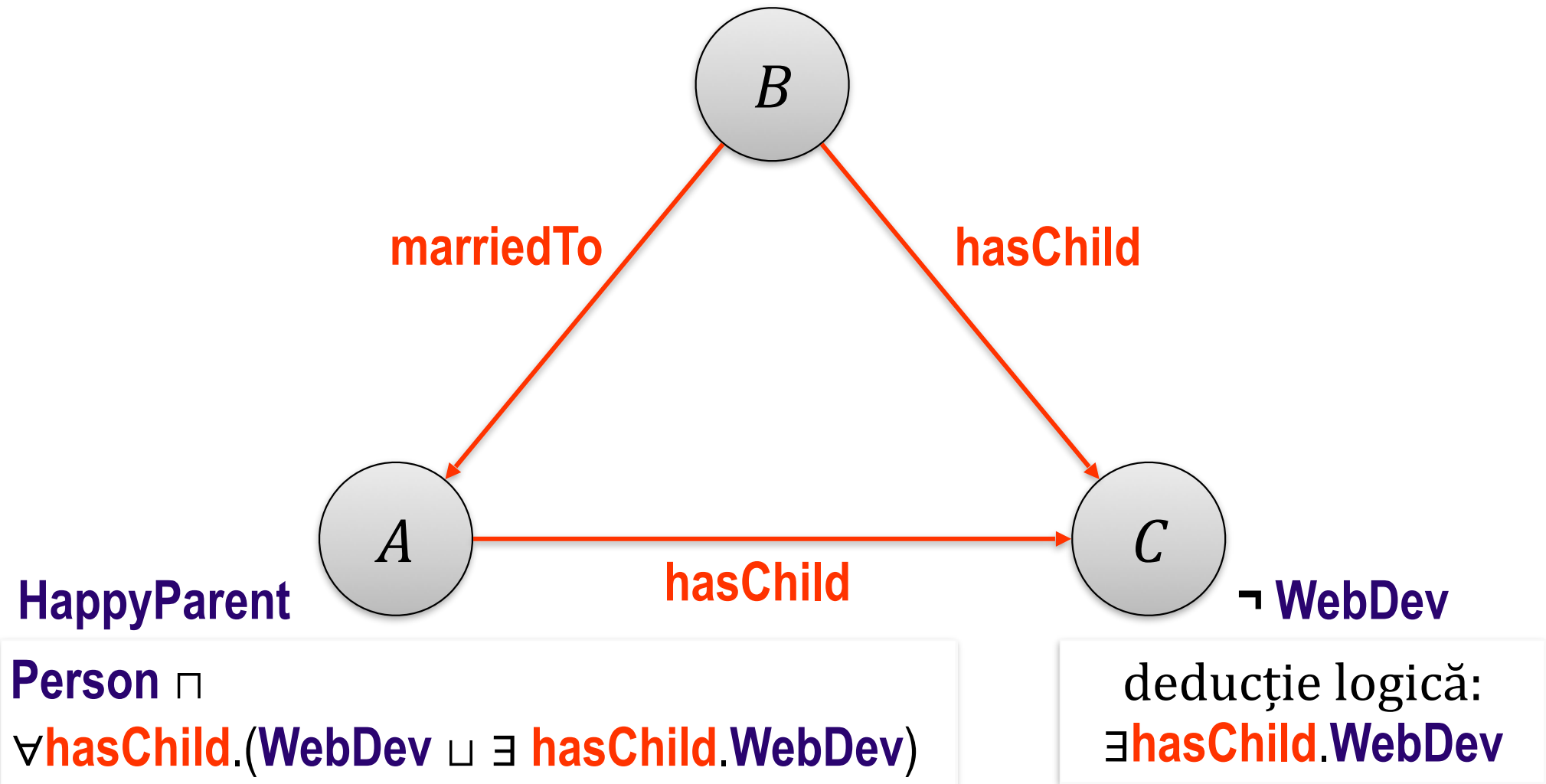
description logics – exemplu



description logics – exemplu



!! Cristina trebuie să aibă un copil dezvoltator Web (altfel, există contradicții – eludarea satisfiabilității)



$TBox:$

$$\text{HappyParent} \sqsubseteq \text{Person} \sqcap \forall \text{hasChild}.(\text{WebDev} \sqcup \exists \text{hasChild}.\text{WebDev})$$
 $\mathcal{T}_1$ $ABox:$

$$\text{Alex} : \text{HappyParent}$$
 $\mathcal{A}_1$

$$\text{Cristina} : \neg \text{WebDev}$$
 $\mathcal{A}_2$

$$\text{Alex}, \text{Cristina} : \text{hasChild}$$
 $\mathcal{A}_3$

$$\text{Bianca}, \text{Cristina} : \text{hasChild}$$
 $\mathcal{A}_4$

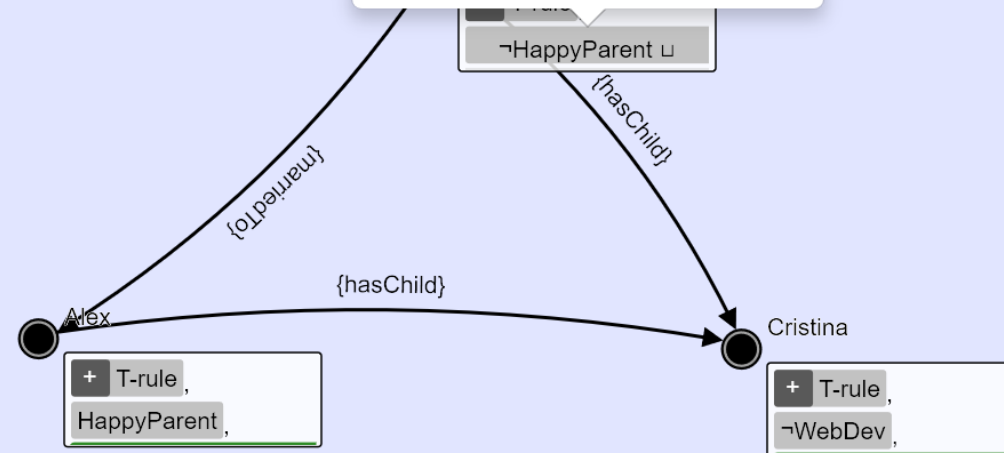
$$\text{Bianca}, \text{Alex} : \text{marriedTo}$$
 $\mathcal{A}_5$ Knowledge base consistency checking of $\mathcal{K}$

Tableau rule

Insert " $\neg \text{HappyParent}$ " or " $\text{Person} \sqcap \forall \text{hasChild}.(\text{WebDev} \sqcup \exists \text{hasChild}.\text{WebDev})$ " into label of Bianca node.



View tableau



SIVA – instrument Web de simulare a execuției algoritmului *tableau*

P. Paulovics, J. Pukancová & M. Homola, *FOMEO 2022*

easychair.org/smart-slide/slide/BIP3 • dai.fmph.uniba.sk/~pukancova/siva/

www.ceur-ws.org/Vol-2211/paper-29.pdf

$\text{Person} \sqcap \forall \text{hasChild} . (\text{WebDev} \sqcup \exists \text{hasChild} . \text{WebDev})$

```
<owl:Class>  
  <owl:intersectionOf rdf:parseType="Collection">  
    <owl:Class rdf:about="#Person"/>  
    <owl:Restriction>  
      <owl:onProperty rdf:resource="#hasChild"/>  
      <owl:toClass rdf:resource="#PersonWithChildWebDev"/>  
    </owl:Restriction>  
  </owl:intersectionOf>  
</owl:Class>  
<owl:Class rdf:ID="PersonWithChildWebDev">  
<owl:unionOf rdf:parseType="Collection">  
  <owl:Class rdf:about="#WebDev"/>  
  <owl:Restriction>  
    <owl:onProperty rdf:resource="#hasChild"/>  
    <owl:hasClass rdf:resource="#WebDev" />  
  </owl:Restriction>  
</owl:unionOf>  
</owl:Class>
```

de la formule exprimate via logicile descrierii
la sintaxa XML a limbajului OWL

description logics

În unele cazuri, este convenabil
ca relațiile să fie modelate separat

baza (graful) de cunoștințe: $\mathcal{K} = \langle \mathcal{T}, \mathcal{R}, \mathcal{A} \rangle$

$\mathcal{T}$ – **TBox** definește doar axiomele vizând clase (terminologia)

$\mathcal{R}$ – **RBox** specifică relațiile (proprietățile, rolurile)

$\mathcal{A}$ – **ABox** include aserțiuni (indivizi)

detalii: G. Singh *et al.*, “Neuro-Symbolic RDF and Description Logic Reasoners: The State-Of-The-Art and Challenges”, *Compendium of Neurosymbolic AI*, IOS Press (2023): arxiv.org/abs/2308.04814

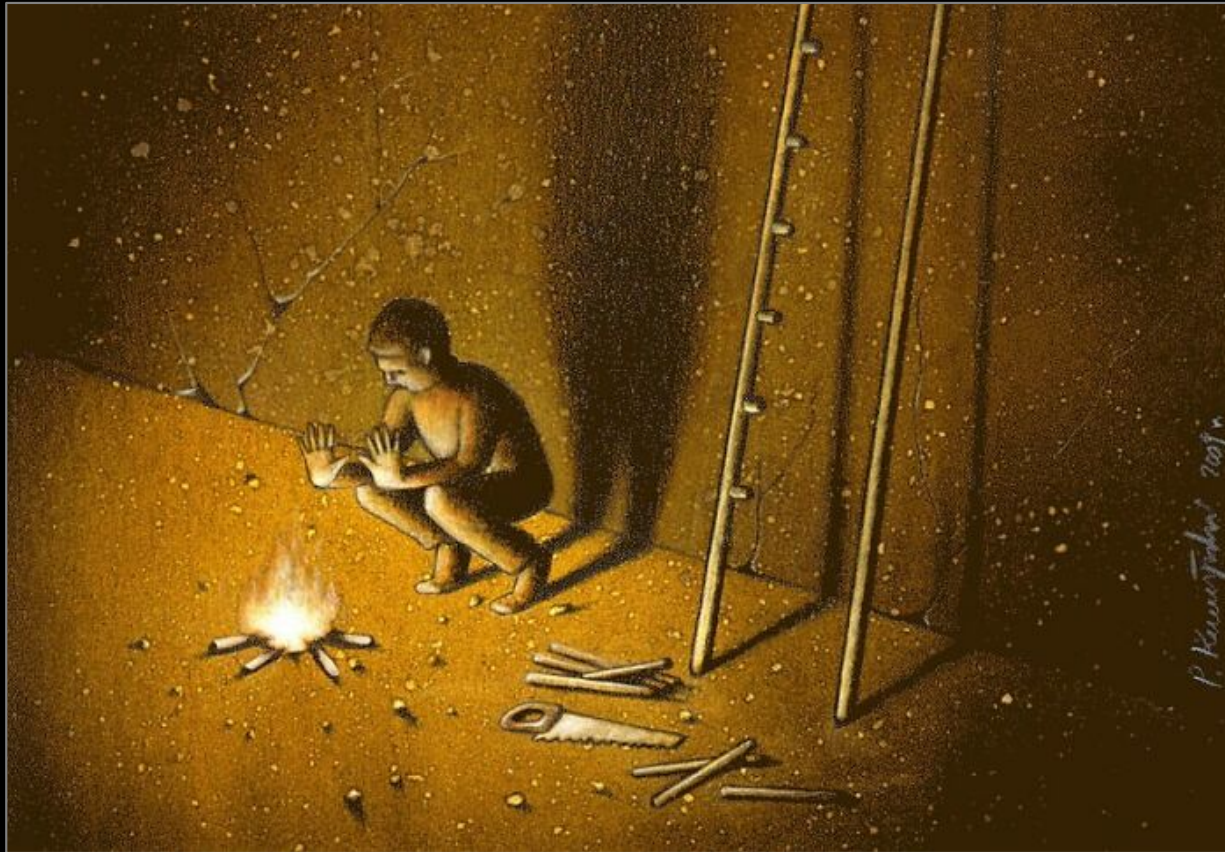
Ontology Name	Number of Axioms						
	Classes	Roles	Individuals	TBox	RBox	ABox	Expressivity
Molecule Role	8849	2	128056	9243	1	128056	$\mathcal{AL}\mathcal{E}+$
XP Uber Anatomy	11427	82	88955	14669	80	88955	$\mathcal{AL}\mathcal{E}\mathcal{H}\mathcal{I}\mathcal{F}+$
XP Plant Anatomy	19145	82	86099	35770	87	86099	$\mathcal{SH}\mathcal{I}\mathcal{F}$
XP Regulators	25520	4	155169	42896	3	155169	$\mathcal{SH}$
Cellular Component	27889	4	163244	47345	3	163244	$\mathcal{SH}$
NCI-1	27652	70	0	46800	140	0	$\mathcal{AL}\mathcal{E}$
Gazetteer	150979	2	214804	167349	2	214804	$\mathcal{AL}\mathcal{E}+$
GALEN-doctored	2748	413	0	3937	799	0	$\mathcal{AL}\mathcal{E}\mathcal{H}\mathcal{I}\mathcal{F}+$
GALEN-undoctored	2748	413	0	4179	800	0	$\mathcal{AL}\mathcal{E}\mathcal{H}\mathcal{I}\mathcal{F}+$
CHEBI	20977	9	243972	38375	2	243972	$\mathcal{AL}\mathcal{E}+$
FMA-Lite	75141	2	46225	119558	3	46225	$\mathcal{AL}\mathcal{E}\mathcal{I}+$
SWEET Phenomena	1728	145	171	2419	239	491	$\mathcal{SHOIN}(\mathcal{D})$
SWEET Numerics	1506	177	113	2184	305	340	$\mathcal{SHOIN}(\mathcal{D})$
Wine	138	17	206	355	40	494	$\mathcal{SHOIN}(\mathcal{D})$
DOLCE-Plans	118	264	27	265	948	68	$\mathcal{SHOIN}(\mathcal{D})$
NCI-2	70576	189	0	100304	290	0	$\mathcal{ALCH}(\mathcal{D})$
FMA-Constitutional	41648	168	85	122695	395	86	$\mathcal{ALCOIF}(\mathcal{D})$

expresivități (specii DL) ale unor ontologii (Boris Motik *et al.*)

RBox = mulțimea axiomelor vizând specificarea rolurilor

e.g., incluziunea, reflexivitatea, simetria,...

www.cs.ox.ac.uk/boris.motik/pubs/msh09hypertableau.pdf



Sistemele de *reasoning* bazate pe logicile descrierii
oferă premisele realizării de inferențe
în mod automat

inferențe

Exemplul 1 – hoțul și călătorul fără bilet
primul fapt: amprentele găsite la locul unui jaf
au identificat ca posibil hoț pe Lödi Gogo

```
</reports/ro/is/2025-12-01/pol07>  
  rdf:type :Jaf ;  
  dc:description "...";  
  :implicaSuspect [  
    rdfs:seeAlso <https://pop.ro/persons/LödiGogo>  
  ].
```

inferențe

Exemplul 1 – hoțul și călătorul fără bilet

alt doilea fapt: în tramvai, a fost găsit un călător fără bilet,
numit Justîn Bip – acesta a fost reținut

```
</reports/ro/is/2025-12-01/pol74>
```

```
  rdf:type :Amenda ;
```

```
  dc:description "...";
```

```
  :prejudiciu [
```

```
    :comisDe :Cetatean ;
```

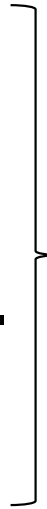
```
    rdfs:seeAlso <https://pop.ro/persons/JustînBip>
```

```
  ].
```

inferențe

Exemplul 1 – hoțul și călătorul fără bilet
poliția deține o aplicație Web ce recurge la
cunoștințe stipulând următoarele:

```
:Jaf      rdfs:subClassOf :Infractiune .  
:Amenda  rdfs:subClassOf :Infractiune .  
:implicaSuspect a owl:ObjectProperty ;  
                rdfs:domain :Infractiune ; rdfs:range :Suspect .  
:Suspect  rdfs:subClassOf :Cetatean .  
:Cetatean rdfs:subClassOf schema:Person .
```

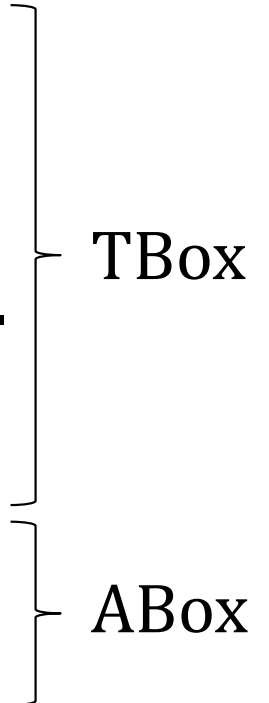


TBox

inferențe

Exemplul 1 – hoțul și călătorul fără bilet
poliția deține o aplicație Web ce recurge la
cunoștințe stipulând următoarele:

```
:Jaf      rdfs:subClassOf :Infractiune .
:Amenda  rdfs:subClassOf :Infractiune .
:implicaSuspect a owl:ObjectProperty ;
               rdfs:domain :Infractiune ; rdfs:range :Suspect .
:Suspect  rdfs:subClassOf :Cetatean .
:Cetatean rdfs:subClassOf schema:Person .
<https://pop.ro/persons/LödiGogo> a schema:Person ;
  owl:sameAs <https://pop.ro/persons/JustinBip> .
```



TBox

ABox

inferențe

Exemplul 1 – hoțul și călătorul fără bilet

concluzie: se poate realiza (automat) inferența
„Hoțul este călătorul prins fără bilet.”

pe baza logicilor descrierii, OWL oferă posibilitatea
de a exprima faptul că două resurse – *e.g.*, clase,
proprietăți, instanțe – sunt identice

inferențe

Exemplul 2 – achiziționarea unui aparat foto digital

☞ *Can I take a photo with this?*

Aplicație Web
recomandând
cumpărături
online

„Trimite-mi catalogul
de produse” (via un API)

```
<NikonD850> a :DSLR ;  
:sensorSize [  
  :width "35.9"^^units:mm ;  
  :height "23.9"^^units:mm ] ;  
:effPixels "45.7"^^pix:megapixel;  
schema:offers [ schema:price ... ]
```

„Iată catalogul...”
(date RDF
în format Turtle)

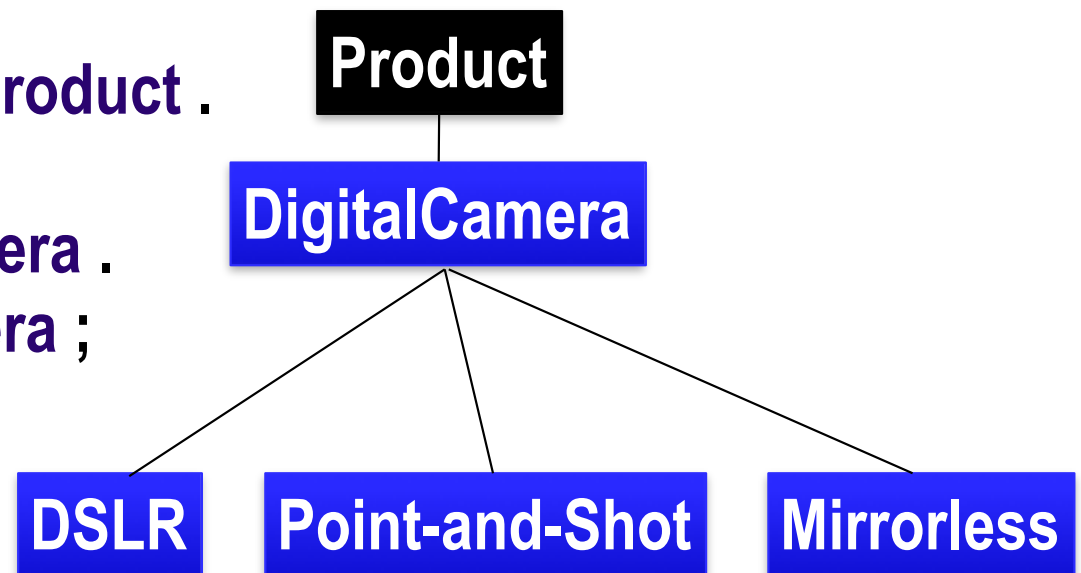
situl Web al
magazinului

inferențe

Exemplul 2 – achiziționarea unui aparat foto digital

aplicația Web consultă o ontologie privitoare la camere digitale de unde se poate determina că DSLR este subclasă a clasei Camera asociată echipamentelor foto

```
:Camera rdfs:subClassOf schema:Product .  
:DSLR a owl:Class ;  
      rdfs:subClassOf :DigitalCamera .  
:takes rdfs:domain :DigitalCamera ;  
      rdfs:range :Photo .  
<NikonD850> a :DSLR .
```



🧠 Can I take a photo with this?

Aplicație Web
recomandând
cumpărături
online

🤖 Yes,
indeed!

„Trimite-mi catalogul
de produse” (via un API)

```
<NikonD850> a :DSLR ;  
:sensorSize [  
  :width "35.9"^^units:mm ;  
  :height "23.9"^^units:mm ] ;  
:effPixels "45.7"^^pix:megapixel;  
schema:offers [ schema:price ... ]
```

„Iată catalogul...”
(date RDF
în format Turtle)

situl Web al
magazinului

inferențe

Exemplul 2 – achiziționarea unui aparat foto digital

ontologiile exprimate în OWL pot conduce la descoperirea dinamică a relațiilor dintre resurse

- pe baza unui sistem de deducție
- via construcții schema.org și/sau RDFa –
pot fi extrase, comparate și recomandate
oferțele disponibile pe diverse situri Web

inferențe

Exemplul 3 – locul de naștere a regelui Kamehameha

fapt: în urma preluării datelor de pe Web, s-a obținut

<<http://www.persoane.info#Kamehameha>> a :Individ ;
:locNastere <<https://www.states.us/Hawaii>> .

<<http://www.persoane.info#Kamehameha>> a :Individ ;
:locNastere <https://world.info/Sandwich_Islands> .

<<http://www.persoane.info#Kamehameha>> a :Individ ;
:locNastere <<https://history.org/Aloha>> .

inferențe

Exemplul 3 – locul de naștere a regelui Kamehameha

ontologia privitoare la persoane indică

faptul că un individ are un unic loc de naștere

- ▶ se poate trage concluzia că *Hawaii, Sandwich Islands* și *Aloha* reprezintă aceeași localitate

OWL oferă suport pentru precizarea cardinalității unei relații dintre o resursă și o alta – în acest caz, exact una

inferențe: exemple

Folosirea unui *reasoner*

– e.g., **HermiT** sau **Pellet** incluse în Protégé –
pentru realizarea de deducții asupra unor aspecte
vizând un model conceptual exprimat în OWL

exemplul #1: proprietari de artefacte multimedia

exemplul #2: rezolvarea șaradei *Zebra Puzzle*

inferențe: studiu de caz #1

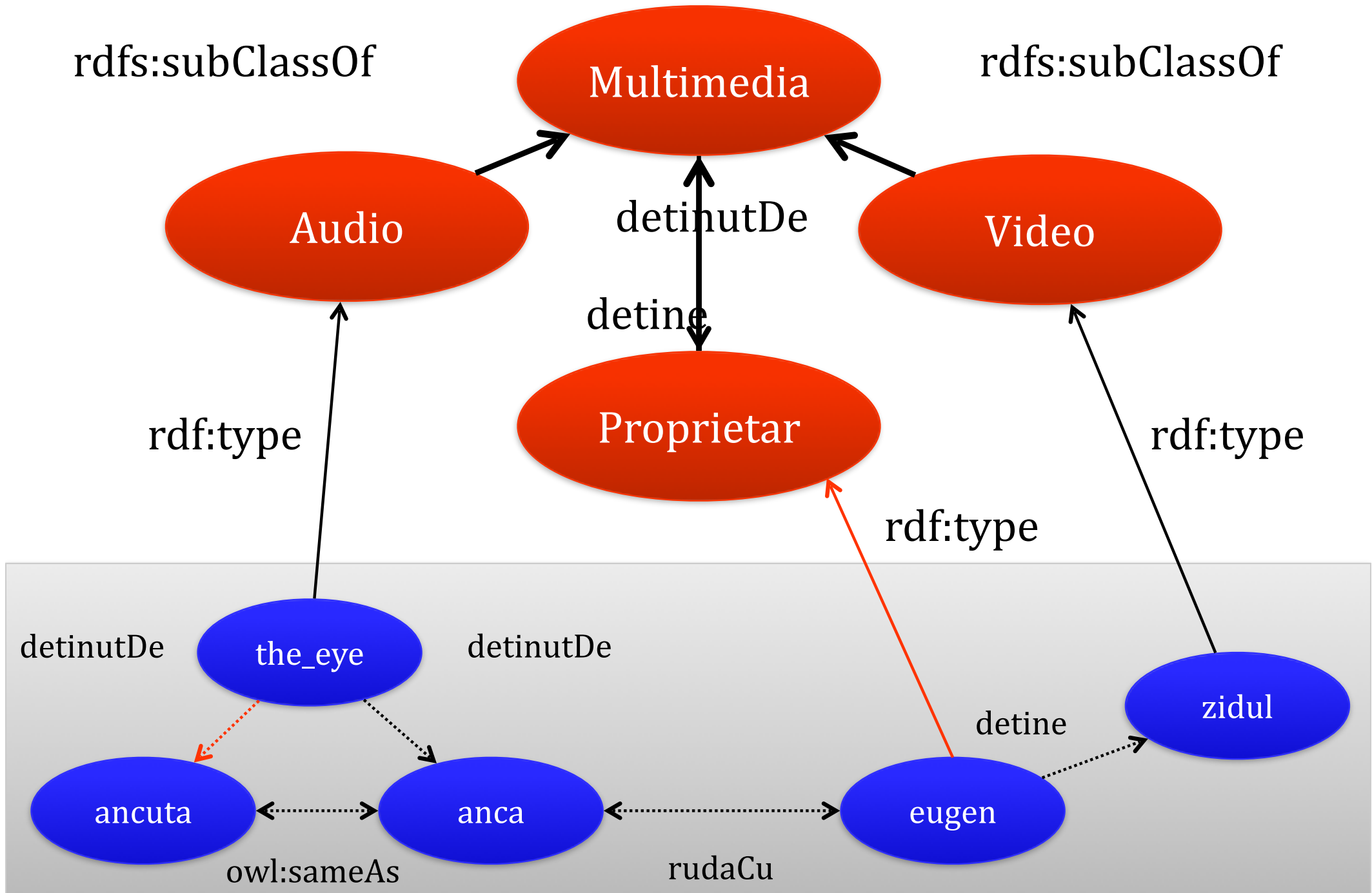
O ontologie – exprimată în OWL – modelând
artefacte multimedia (audio, video)
deținute de persoane caracterizate
prin diverse proprietăți: gen, vârstă, relație de rudenie

spațiul de nume folosit:

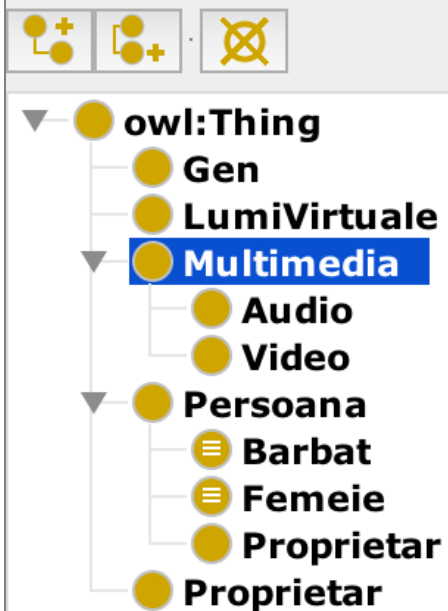
www.semanticweb.org/ontologies/2008/10/SWOnto.owl



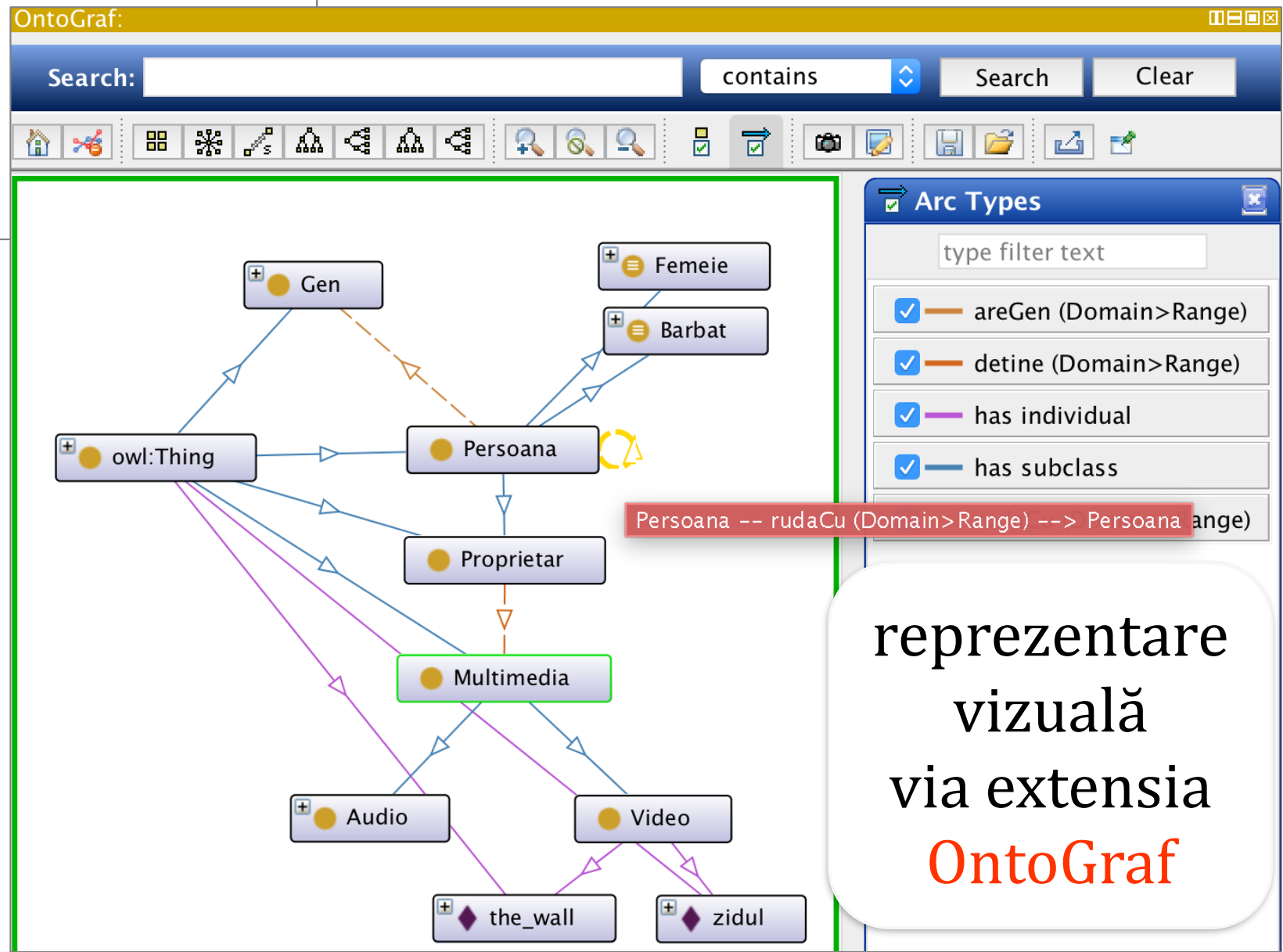
de consultat
arhiva aferentă
acestei prelegeri



Class hierarchy: Multimedia



editarea și vizualizarea ontologiei
cu instrumentul **Protégé**



Individuals: ancuta

Annotations

Annotations: ancuta

Annotations +

`rdfs:comment` [type: xsd:string]

'Ancuta' este un individ identic cu 'Anca'.

Description: ancuta

Types +

- `owl:Thing`
- Femeie
- Proprietar

Same Individual As +

`ancuta`

Different Individuals +

Property assertions: ancuta

Object property assertions +

- `rudaCu eugen`
- `rudaCu ancuta`
- `rudaCu anca`
- `detine the_wall`
- `detine the_eye`
- `areGen Feminin`

Data property assertions +

- `areVarsta "24"^^xsd:byte`

deducții automate cu **HermiT**
pe baza faptelor exprimate

inferențe: studiu de caz #1

Realizarea de raționamente automate
– interogări (*queries*) DL –
cu ajutorul expresiilor de clasă (*class expression*)

specificate în sintaxa Manchester (2012)
www.w3.org/TR/owl2-manchester-syntax/

DL query:

Query (class expression)

Proprietar

Execute Add to ontology

Query results

Superclasses (2 of 2)

-  **Persoana** 
-  **owl:Thing** 

Instances (3 of 3)

-  **anca** 
-  **ancuta** 
-  **eugen** 

Query for

- ☐ Direct superclasses
- ☒ Superclasses
- ☐ Equivalent classes
- ☐ Direct subclasses
- ☐ Subclasses
- ☒ Instances

Explain inference

deducerea faptelor referitoare la clasa **Proprietar**
(în acest caz, supraclasele și indivizii aparținând clasei)

☒ Show regular justifications☒ All justifications☐ Show laconic justifications☐ Limit justifications to

2



Explanation 1

☐ Display laconic explanation

Explanation for: ancuta Type Proprietar

- 1) detine **Domain** Proprietar
- 2) detine **InverseOf** detinutDe
- 3) the_wall detinutDe ancuta

explicarea de către sistem a
raționamentelor automate
aici, determinarea faptului
:ancuta rdf:type :Proprietar

Explanation 2

☐ Display laconic explanation

Explanation for: ancuta Type Proprietar

- | | | |
|------------------------------------|-----------------------------|---|
| 1) anca SameAs ancuta | In 1 other justifications | ? |
| 2) anca detine the_wall | In NO other justifications | ? |
| 3) detine Domain Proprietar | In ALL other justifications | ? |

Explanation 3

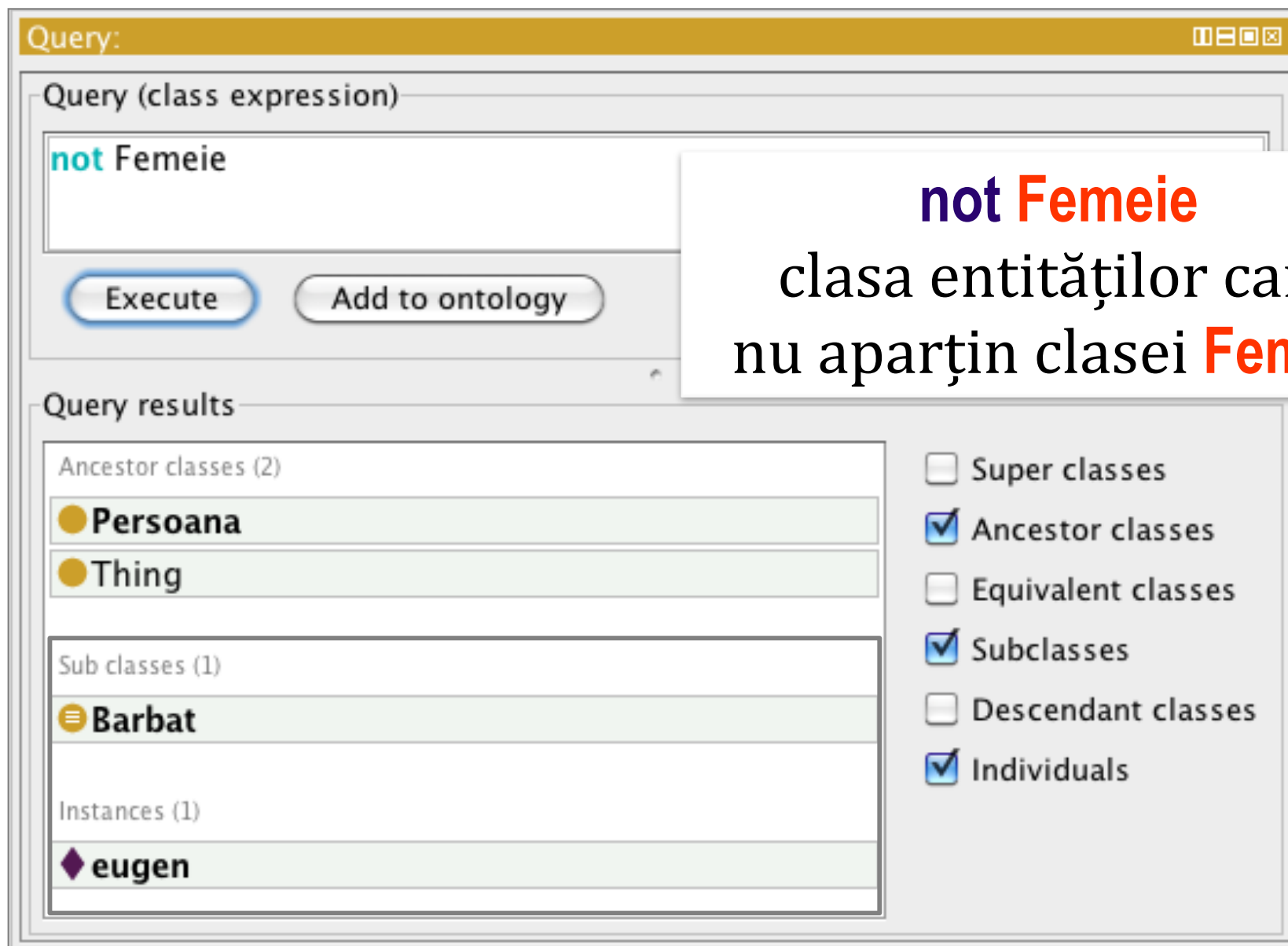
☐ Display laconic explanation

Explanation for: ancuta Type Proprietar

- | | | |
|--------------------------------------|-----------------------------|---|
| 1) anca SameAs ancuta | In 1 other justifications | ? |
| 2) detine Domain Proprietar | In ALL other justifications | ? |
| 3) detine InverseOf detinutDe | In 1 other justifications | ? |
| 4) the_eye detinutDe anca | In NO other justifications | ? |

Inferred

Matthew Horridge, Bijan Parsia, Ulrike Sattler, "Laconic and Precise Justifications in OWL", *ISWC 2008* – owl.cs.manchester.ac.uk/research/explanation/



not Femeie
clasa entităților care
nu aparțin clasei **Femeie**

obținerea claselor complementare cu una specificată

Query:



Query (class expression)

rudaCu **some** Femeie

Execute

Add to ontology

Query results

Ancestor classes (2)

● Persoana

● Thing

Descendant classes (1)

☰ Femeie

Instances (3)

◆ ancuta

◆ anca

◆ eugen

Usage: rudaCu

Show: ☒ this ☒ disjoints

Found 12 uses of rudaCu

▼ ◆ ancuta

◆ ancuta rudaCu eugen

▼ ■ rudaCu

■ ObjectProperty: rudaCu

■ rudaCu Domain Persoana

■ Symmetric: rudaCu

■ Reflexive: rudaCu

■ rudaCu Range Persoana

rudaCu some Femeie

entitățile care sunt rude cu măcar o femeie
(utilizarea operatorului existențial **some**)

☐ Inverse functional

☐ Transitive

☒ Symmetric

☐ Asymmetric

☒ Reflexive

☐ Irreflexive

SubProperty Of +

Inverse Of +

■ rudaCu

Inferred

Domains (intersection) +

● Persoana

Ranges (intersection) +

● Persoana

DL query:

Query (class expression)

Multimedia **and** detinutDe **some** Persoana

Execute

Add to ontology

Query results

Direct superclasses (1 of 1)



Multimedia



Instances (3 of 3)



the_eye



the_wall



zidul



Query for

- ☒ Direct superclasses
- ☐ Superclasses
- ☐ Equivalent classes
- ☐ Direct subclasses
- ☐ Subclasses
- ☒ Instances

**Multimedia and
detinutDe some Persoana**
clasa resurselor multimedia
deținute de o persoană

specificarea intersecțiilor de clase via operatorul **and**
+ exprimarea intervalului (*range*-ului) unei relații

Query (class expression)

Femeie **and** rudaCu **only** Barbat

Execute

Add to ontology

Femeie and rudaCu only Barbat
recurgerea la operatorul universal **only**

Query results

Equivalent classes (1 of 1)

owl:Nothing

Query for

☒ Direct superclasses

☐ Superclasses

☐ Equivalent classes

☐ Direct subclasses

☐ Subclasses

☐ Instances

☐ filters

☐ contains

☐ play owl:Thing
(superclass results)

☐ Display owl:Nothing
(in subclass results)

Explanation for owl:Nothing EquivalentTo Femeie and (rudaCu only Barbat)

☒ Show regular justifications

☐ Show laconic justifications

☒ All justifications

☐ Limit justifications to

2

Explanation 1

☐ Display laconic explanation

Explanation for: owl:Nothing EquivalentTo Femeie and (rudaCu only Barbat)

Reflexive: rudaCu

Barbat **DisjointWith** Femeie

justificarea faptului că nu există
rezultate (s-a obținut **owl:Nothing**)

OK

inferențe: studiu de caz #1

DL query:

Query (class expression)

Video **and** detinutDe **value** anca

Execute Add to ontology

Query results

Direct superclasses (1 of 1)

 **Video**

Instances (1 of 1)

 **the_wall**

Video and detinutDe value anca
instance din clasa resurselor video
deținute de individul Anca

specificarea de valori via operatorul **value**
în contextul relațiilor

Query (class expression)

(Audio **or** Video **or** LumiVirtuale) **and** detinutDe **some** Femeie

Execute Add to ontology

Query results

Ancestor classes (3)

- Multimedia
- Persoana
- Thing

Descendant classes (0)

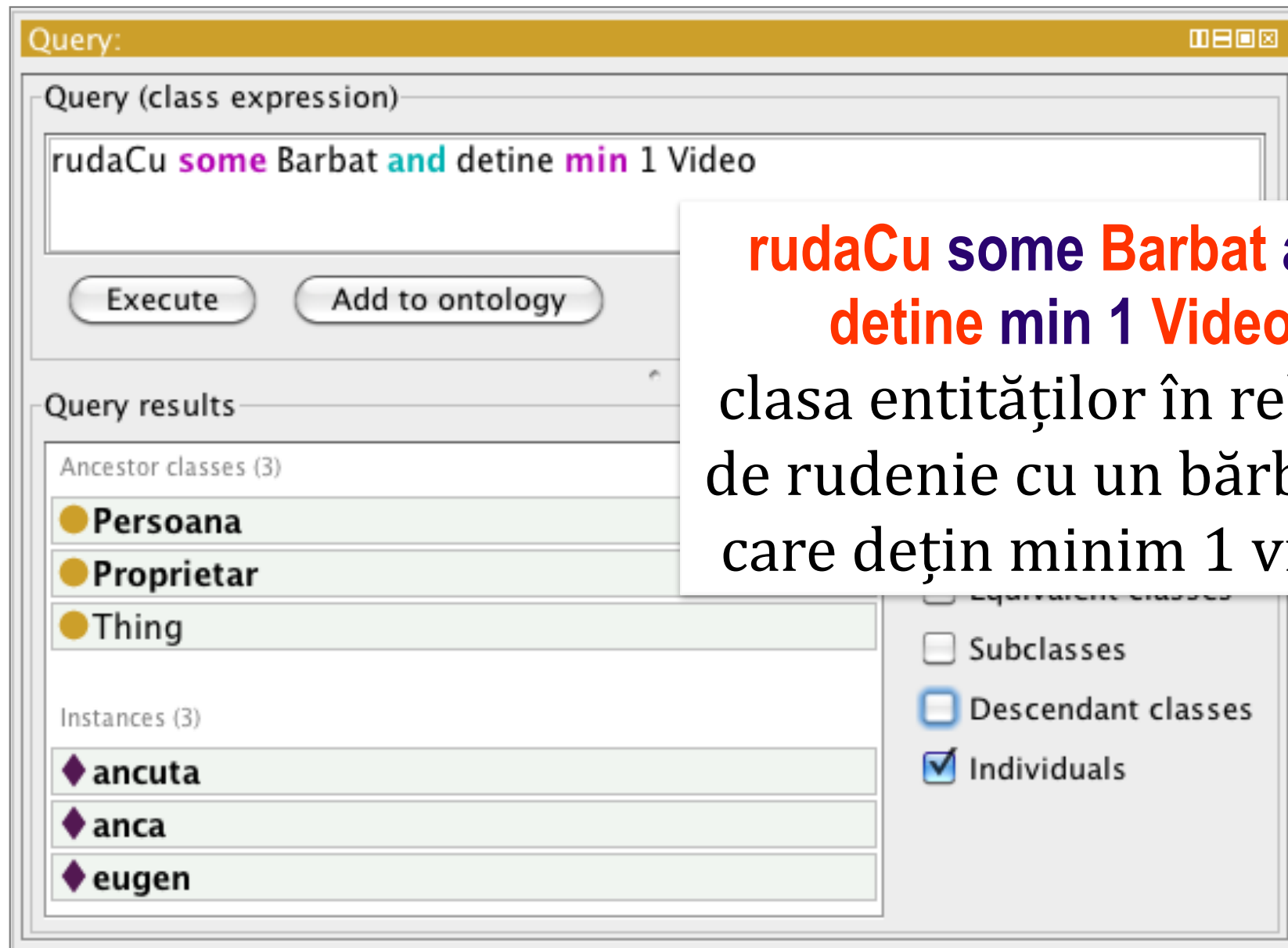
Instances (2)

- ◆ the_eye
- ◆ the_wall

☐ Super classes
☒ Ancestor classes
☐ Equivalent classes
☐ Subclasses
☒ Descendant classes
☒ Individuals

(Audio **or** Video **or** LumiVirtuale)
and detinutDe **some** Femeie

expresii de clasă mai complexe
operatorul **or** desemnează reuniunea claselor



**rudaCu some Barbat and
detine min 1 Video**
clasa entităților în relație
de rudenie cu un bărbat și
care dețin minim 1 video

exprimarea cardinalității via operatorii **min**, **max** și **exactly**

Query (class expression)

Multimedia **and** detinutDe **some** (rudaCu **value** ancuta **and not** (areGen **value** Feminin))

Execute **Add to ontology**

Query results

Direct superclasses (1 of 1)

 **Multimedia** 

Instances (1 of 1)

 **zidul** 

Query for

- ☒ Direct superclasses
- ☐ Superclasses
- ☐ Equivalent classes
- ☐ Direct subclasses
- ☐ Subclasses
- ☒ Instances

expresii de clasă compuse:

Multimedia and detinutDe some (rudaCu value ancuta and not (areGen value Feminin))

obținerea artefactelor multimedia care sunt deținute de entități rude cu Ancuța și care n-au genul feminin

Explanation for zidul Type Multimedia and (detinutDe some ((not (Femeie)) and (rudaCu value ancuta)))

☒ Show regular justifications
 ☐ All justifications

☐ Show laconic justifications
 ☒ Limit justifications to

Explanation 1
 ☐ Display laconic explanation

Explanation for: zidul Type Multimedia and (detinutDe some ((not (Femeie)) and (rudaCu value ancuta)))		
1)	eugen detine zidul	In ALL other justifications ?
2)	detine InverseOf detinutDe	In ALL other justifications ?
3)	Symmetric: rudaCu	In ALL other justifications ?
4)	Barbat DisjointWith Femeie	In ALL other justifications ?
5)	eugen Type Barbat	In ALL other justifications ?
6)	detine Range Multimedia	In NO other justifications ?
7)	ancuta rudaCu eugen	In ALL other justifications ?

Fie interogarea DL: **Multimedia and detinutDe some (rudaCu value ancuta and not Femeie)**

Rezultatul obținut e același cu cel al deducției anterioare?
Ce se poate afirma în privința eficienței?

inferențe: studiu de caz #2

Exprimarea la nivel ontologic a șaradei *Zebra Puzzle* publicată în 1962 (cunoscută și drept *Einstein's riddle*)

modelare – în termenii logicii descrierii –
realizată de Denis Ponomaryov (2013)

persons.iis.nsk.su/en/ponom/ontologies

Ontology header:

Ontology IRI `http://persons.iis.nsk.su/files/persons/pages/einsteins_riddle.owl`

Ontology Version IRI e.g. `http://persons.iis.nsk.su/files/per`

Annotations

dc:description [type: xsd:string]

1. There are five houses.
2. The Englishman lives in the red house.
3. The Spaniard owns the dog.
4. Coffee is drunk in the green house.
5. The Ukrainian drinks tea.
6. The green house is immediately to the right of the ivory house.
7. The Old Gold smoker owns snails.
8. Kools are smoked in the yellow house.
9. Milk is drunk in the middle house.
10. The Norwegian lives in the first house.
11. The man who smokes Chesterfields lives in the house next to the blue house.
12. Kools are smoked in a house next to the house where the Norwegian lives.
13. The Lucky Strike smoker drinks orange juice.
14. The Japanese smokes Parliaments.
15. The Norwegian lives next to the blue house.

Now, who drinks water? Who owns the zebra?

Ontology imports Ontology Prefixes General class axioms

Ontology prefixes:

Prefix	Value
dc	http://purl.org/dc/terms/
einsteins_riddle	http://persons.iis.nsk.su/files/persons/pages/einsteins_riddle.owl
owl	http://www.w3.org/2002/07/owl#

Ontology metrics:

Metrics	
Axiom	150
Logical axiom count	92
Declaration axioms count	46
Class count	8
Object property count	6
Data property count	0
Individual count	30
DL expressivity	ALCOIF

Class axioms	
SubClassOf	6
EquivalentClasses	6
DisjointClasses	0
GCI count	0
Hidden GCI Count	6

DL Expressivity

ALCOIF

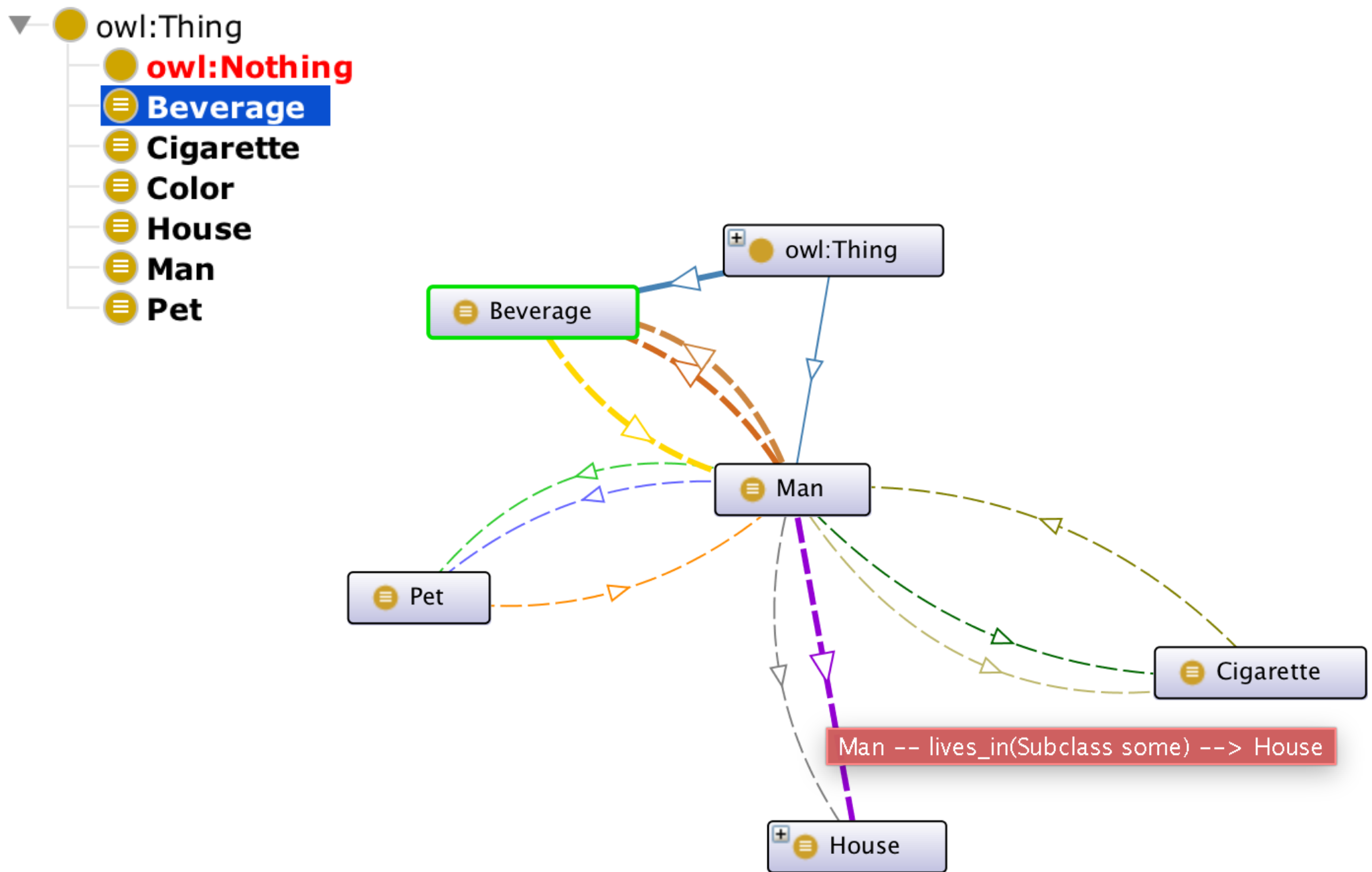
Symbol key

Attributive language. This is the base language which allows:

- Atomic negation (negation of concepts that do not appear on the left hand side of axioms)
- Concept intersection
- Universal restrictions
- Limited existential quantification (restrictions that only have fillers of Thing)

AL

expresivitatea modelului conceptual corespunde logicii
ALCOIF (concept satisfiability: *NExpTime-complete*)



principalele concepte (clase) ale ontologiei modelate

DL query:

Query (class expression)

drinks **value** water

Execute Add to ontology

Query results

Direct superclasses (1 of 1)

 **Man** 

Subclasses (0 of 1)

Instances (1 of 1)

 **Norwegian** 

Query for

- ☒ Direct superclasses
- ☐ Superclasses
- ☐ Equivalent classes
- ☐ Direct subclasses
- ☒ Subclasses
- ☒ Instances

Explain inference

răspunsul la întrebarea “*Who drinks water*”

– exprimat via interogarea DL **drinks value water** – furnizat de instrumentul de efectuare a raționamentelor automate: individul Norwegian

Individuals: Norwegian



- blue
- Chesterfields
- coffee
- dog
- Englishman
- fox
- green
- horse
- house-1
- house-2
- house-3
- house-4
- house-5
- ivory
- Japanese
- Kools
- Lucky_Strike
- milk
- Norwegian**
- Old_Gold
- orange_juice
- Parliaments
- red
- snails
- Spaniard
- tea
- Ukrainian
- water
- yellow
- zebra

Annotations Usage

Usage: Norwegian

Show: ☒ this ☒ different

Found 71 uses of Norwegian

▼ blue

◆ **DifferentIndividuals:** Chesterfields, Englishman, Japanese, Kools, Lucky_Strike, M

▼ Chesterfields

◆ **DifferentIndividuals:** Chesterfields, Englishman, Japanese, Kools, Lucky_Strike, M

▼ coffee

Description: Norwegian

Types +

● lives_in some
((right_to some
(has_color value
blue)) or (inverse
(right_to) some
(has_color value

● owl:Thing

☰ Man

Same Individual As +

Different Individuals +

◆ Chesterfields,
Englishman,
Japanese, Kools,
Lucky_Strike,
Old_Gold,

Property assertions: Norwegian

Object property assertions +

■ lives_in house-1

■ lives_in house-1

■ drinks water

■ owns fox

■ smokes Kools

Data property assertions +

fapte deduse
de *reasoner*

DL query:

Query (class expression)

Man **and** owns **value** zebra

Execute Add to ontology

Query results

Direct superclasses (1 of 1)

 **Man** 

Subclasses (0 of 1)

Instances (1 of 1)

 **Japanese** 

Query for

- ☒ Direct superclasses
- ☐ Superclasses
- ☐ Equivalent classes
- ☐ Direct subclasses
- ☒ Subclasses
- ☒ Instances

Explain inference

interogarea “*Who owns the zebra*”

- în sintaxa Manchester: **Man and owns value zebra** –
oferă ca răspuns entitatea Japanese

Taxonomy

Reasoner

Materialized Graph

XML Preview

SPARQL

Annotations

Reference Explorer

Tools

CNL Diagram

Diagram

SWRL Debug

Run Debugger

SWRL Debugger

Active Rules

Run Active Rules

Active Rules Debug...

Draw Graph Ontology

Enumerate Universe

R Plugins

Develop R Plugins

Document

Title: 'Young persons knowing some students'.

Author: 'Dr. Sabin Buraga'.

Namespace: 'http://www.info.uaic.ro/wade/young-persons'.

Comment: 'TBox'.

Every young-person has-age lower-than 40.

Every young-person is a person.

Every person is a living-thing.

Every male is a person.

Every female is a person.

No male is a female.

Every student is a person.

X knows Y if-and-only-if Y known-by X.

Every young-person knows a student.

Comment: 'ABox'.

Ana is a female.

Ana is a young-person that knows Dan.

Cristina known-by Ana.

Dan is a male.

Cristina is a young-person that knows Dan.

George knows a student.

Taxonomy Tree

- thing
 - living-thing
 - person
 - female
 - Ana
 - male
 - Dan
 - student
 - young-person
 - Ana
 - Cristina
- nothing
- relation
 - know
 - known-by
- attribute
 - have-age

Reasoner

Who-Or-What is a young-person that knows a student?

<?> is a young-person and knows a student. Every <?> is a young

Ana

Cristina

young-person

person

Ready

OK

DL: male $\sqsubseteq \neg(\text{female})$

Fluent Editor – deducții pentru ontologii
modelate în limbaj natural controlat
www.cognitum.eu/semantics/FluentEditor/

"No male is a female": $\text{male} \sqsubseteq \neg(\text{female})$

Classes

- Academic Discipline
 - Engineering
 - + Chemical Engineering
 - + Civil Engineering
 - + Cyberinfrastructure
 - + Electrical Engineering
 - + Materials Engineering
 - + Materials Science
 - Mechanical Engineering
 - Software Engineering
 - Systems Engineering
 - + Formal Science
 - + Natural Science
 - + Social Science

ADCAD - Arctic Data Center Academic Disciplines Ontology

[Overview](#)
[Browse](#)
[DLQuery](#)
[SPARQL](#)
[Download](#)

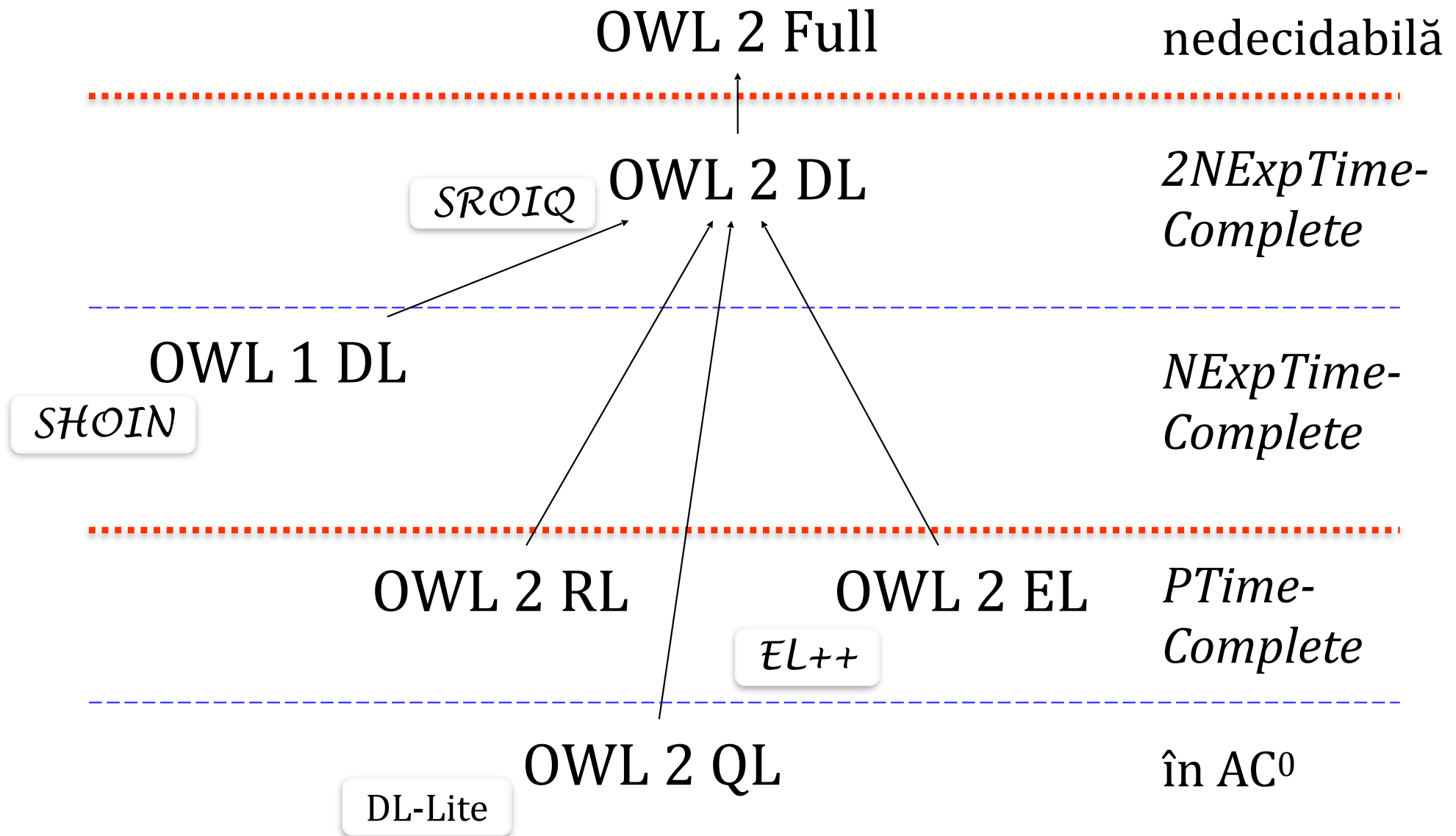
[Subclasses](#)
[Sub and Equivalent](#)
[Equivalent](#)
[Superclasses](#)
[Super and Equivalent](#)

OWLClass	Label	Definition
<https://purl.dataone.org/odo/ADCAD_00072>	Chemical Engineering	
<https://purl.dataone.org/odo/ADCAD_00073>	Civil Engineering	
<https://purl.dataone.org/odo/ADCAD_00062>	Cyberinfrastructure	
<https://purl.dataone.org/odo/ADCAD_00056>	Electrical Engineering	
<https://purl.dataone.org/odo/ADCAD_00050>	Engineering	
<https://purl.dataone.org/odo/ADCAD_00052>	Materials Engineering	
<https://purl.dataone.org/odo/ADCAD_00053>	Materials Science	
<https://purl.dataone.org/odo/ADCAD_00074>	Mechanical Engineering	
<https://purl.dataone.org/odo/ADCAD_00058>	Software Engineering	

Object Properties

raționamente automate (*DLQuery*) efectuate de o aplicație
 Web oferind acces (via API) la o colecție de ontologii
 biomedicale – **AberOWL**: aber-owl.net/ontology/

inferențe: owl2



vezi clasele de complexitate la complexityzoo.net

Pentru aspecte specifice OWL2, a se studia:

D. Lembo, R. Rosati, *Semantic Technologies
for Data Integration using OWL2 QL*, ESWC 2009
www.dis.uniroma1.it/~rosati/Lembo-Rosati-ESWC-09-tutorial.pdf

R. Stevens *et al.*, *Manchester Family History
Advanced OWL Tutorial*, 2015
owl.cs.manchester.ac.uk/publications/talks-and-tutorials/fhkbtutorial/

concluzii

Obținerea de noi cunoștințe de interes are loc
prin intermediul unor procese deductive riguroase
(*i.e.* raționamente automate)
via logicile descrierii – *deductive knowledge*

concluzii

Proprietăți formale înțelese în profunzime
(complexitate, decidabilitate)

Algoritmi de raționament automat eficienți

Sisteme de deducție (*reasoning*)
având implementări optimizate

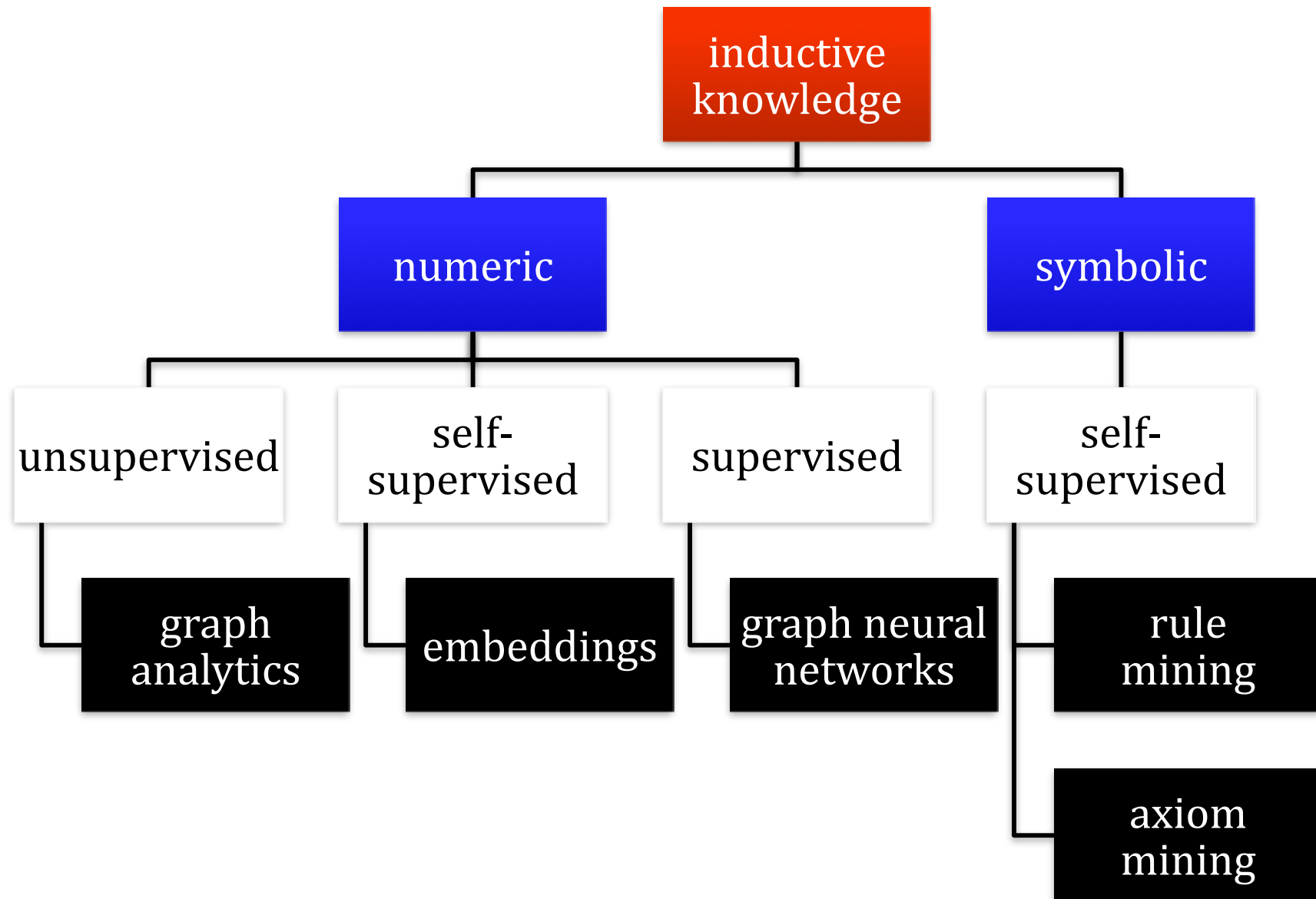
concluzii

Complementar, se poate recurge la
procesul inductiv – *inductive knowledge*

*“acquiring knowledge involves generalizing patterns from
a given set of input observations, which can then be used
to generate novel but potentially imprecise predictions”*

A. Hogan *et al.*, *Knowledge Graphs*,
Morgan & Claypool, 2021 – kgbook.org

concluzii





de la OWL 2 la logicile descrierii
baze de cunoștințe
efectuarea de raționamente automate



episodul viitor: **vacanță...** 🎄